



MANIPAL
ACADEMY of HIGHER EDUCATION
(Institution of Eminence Deemed to be University)

MANIPAL SCHOOL OF INFORMATION SCIENCES
(A Constituent unit of MAHE, Manipal)

AI - Based Intrusion Detection System for IoT Networks Using Machine Learning and Deep Learning

Reg. Number	Name	Branch
261100610001	B. Nitin Sri Karthikeya	CSE
261100610017	E. Dhanush Rajkumar	CSE
261100610019	M.B. Usha	CSE

Under the guidance of

Raghudathesh G P
Assistant Professor - Senior Scale,
Manipal School of Information Sciences,
MAHE, MANIPAL

24/09/2026



MANIPAL SCHOOL OF INFORMATION SCIENCES
MANIPAL
(A constituent unit of MAHE, Manipal)

TABLE OF CONTENTS

S.NO	TITLE	Page No
1	INTRODUCTION	1
	1.1. Background of The Study	1
	1.2. Problem Statement	3
	1.3. Motivation	4
	1.4. Proposed Solution	5
2	OBJECTIVES	7
3	LITERATURE SURVEY	8
	3.1. Introduction	8
	3.2. Summary of Existing Research Papers	9
	3.3. Selected Research Paper	11
	3.4. Research Gap	14
4	SPECIFICATIONS	16
	4.1. Hardware Specifications	16
	4.2. Software Specifications	16
	4.3. Programming Languages and Libraries	17
	4.4. Dataset Specification	18
	4.5. Performance Evaluation Metrics	18
5	ARCHITECTURE	19
	5.1. System Overview	19
	5.2. System Architecture	20
	5.3. System Modules	22
	5.3.1. Dataset Collection	22

	5.3.2. Data Preprocessing	23
	5.3.3. Exploratory Data Analysis	23
	5.3.4. Feature Selection using PCC	24
	5.3.5. Random Forest (RF) Module	25
	5.3.6. Artificial Neural Network (ANN) Module	25
	5.3.7. Convolutional Neural Network (CNN) Module	26
	5.3.8. Performance Evaluation	26
6	WORK DONE	27
	6.1. Implementation Workflow	27
	6.2. Dataset Collection	27
	6.3. Dataset Merging	28
	6.4. Dataset Sampling	29
	6.5. Data Cleaning	30
	6.6. Exploratory Data Analysis (EDA)	31
	6.6.1. Dataset Overview	32
	6.6.2. Statistical Summary	32
	6.6.3. Class Distribution Analysis	33
	6.6.4. Correlation Analysis	33
	6.6.5. Observations from EDA	34
	6.7. Binary Classification	35
	6.8. Pearson Correlation Coefficient (PCC) Feature Selection	36
	6.8.1. PCC Implementation	37
	6.8.2. Feature Ranking	37

	6.8.3. Selected Features	38
	6.8.4. Observations	38
	6.9. Random Forest	38
	6.10. Artificial Neural Network (ANN)	40
	6.11. Convolutional Neural Network (CNN)	42
7	RESULTS AND DISCUSSION	45
	7.1. CNN	45
	7.1.1. Architecture	45
	7.1.2. Hyperparameter Configuration	45
	7.1.3. Training Performance	46
	7.1.4. Cross Validation Results	47
	7.1.5. Quantitative Performance	47
	7.1.6. ROC Analysis	48
	7.1.7. Confusion Matrix	49
	7.1.8. Average Metric Comparison	49
	7.1.9. t – SNE	50
	7.1.10. Discussion	50
	7.2. ANN	51
	7.2.1. Architecture	51
	7.2.2. Hyperparameter Configuration	51
	7.2.3. Training Accuracy	51
	7.2.4. Training Loss	52
	7.2.5. Fold Accuracy	52

	7.2.6. Performance Metrics	53
	7.2.7. ROC Curve	53
	7.2.8. Confusion Matrix	54
	7.2.9. Metric Comparison	54
	7.2.10. Discussion	55
	7.3. Random Forest	55
	7.3.1. Hyperparameter	55
	7.3.2. Fold Accuracy	56
	7.3.3. Experiment Summary	56
	7.3.4. Average ROC	57
	7.3.5. Average Confusion Matrix	58
	7.3.6. Metric Comparison	59
	7.3.7. Discussion	59
	7.4. Comparison	60
	7.4.1. Comparative Discussion	60
8	CONCLUSION	61
9	SCOPE FOR FUTURE WORK	62
10	REFERENCES	6

LIST OF FIGURES

S.No	Fig.No	Title	Pg.No
1	Figure 5.1	Architecture Diagram	20
2	Figure 6.1	Dataset Sampling and Sampled Dataset Generation	30
3	Figure 6.2	Output of Data Cleaning	31
4	Figure 6.3	Dataset Information	32
5	Figure 6.4	Statistical Summary Output	32
6	Figure 6.5	Binary Class Distribution	33
7	Figure 6.6	Correlation Matrix	34
8	Figure 7.1	CNN Architecture	45
9	Figure 7.2	Hyperparameters	45
10	Figure 7.3	Training and Validation Accuracy over epochs	46
11	Figure 7.4	Training and Validation Loss over epochs	46
12	Figure 7.5	Accuracy obtained across 10 Folds	47
13	Figure 7.6	Average ROC Curve across the 10-Fold Curve	48
14	Figure 7.7	Average Confusion Matrix over 10 Validation Folds	49
15	Figure 7.8	Metric Comparison	49
16	Figure 7.9	Two-dimensional t-SNE Visualization of the Binary Intrusion Dataset	50
17	Figure 7.10	ANN Architecture	51
18	Figure 7.11	Hyperparameters	51
19	Figure 7.12	Average Training and Validation across 10-Fold Cross Validation	51

20	Figure 7.13	Average Training and Validation Loss over 10-Fold Cross Validation	52
21	Figure 7.14	Fold Accuracy over 10 Folds	52
22	Figure 7.15	Evaluation Metrics	53
23	Figure 7.16	Average ROC Curve	53
24	Figure 7.17	Average Confusion Matrix over 10 Folds	54
25	Figure 7.18	Metric Comparison	54
26	Figure 7.19	Accuracy over 10 Folds	56
27	Figure 7.20	Average ROC Curve of Random Forest	57
28	Figure 7.21	Average Confusion Matrix for Random Forest	58
29	Figure 7.22	Average Metric Comparison	59

LIST OF TABLES

S.No	Table No.	Title	Pg.No
1	Table 4.1	Hardware Specifications	16
2	Table 4.2	Software Specifications	17
3	Table 4.3	Programming Languages and Libraries	17
4	Table 6.1	Binary Label Classification	36
5	Table 6.2	Binary Class Distribution	36
6	Table 6.3	Libraries Used for Random Forest	39
7	Table 6.4	Libraries Used for ANN	41
8	Table 6.5	Libraries Used for CNN	43
9	Table 7.1	Evaluation Metrics	47
10	Table 7.2	Hyperparameters for Random Forest	55
11	Table 7.3	Evaluation Metrics	56
12	Table 7.4	Model Comparision	60

ABBREVIATIONS

AI	Artificial Intelligence
ANN	Artificial Neural Network
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CSV	Comma-Separated Values
DL	Deep Learning
EDA	Exploratory Data Analysis
FPR	False Positive Rate
GPU	Graphics Processing Unit
IDS	Intrusion Detection System
IoT	Internet of Things
ML	Machine Learning
PCC	Pearson Correlation Coefficient
RF	Random Forest
ROC	Receiver Operating Characteristic
SSM	State Space Model
TPR	True Positive Rate
UI	User Interface

1. INTRODUCTION

1.1. BACKGROUND OF THE STUDY

Internet of Things (IoT) has turned out to be one of the revolutionary technologies in the contemporary digital age that facilitates communications between physical objects via the Internet. IoT integrates several types of smart devices such as sensors, actuators, wearables, household appliances, industrial machines, and medical equipment that can communicate, process, and exchange data with little human participation. Due to the fast development of wireless communication, cloud computing, edge computing, and artificial intelligence, IoT implementation has been facilitated in many industries. Therefore, IoT has become an important part of a variety of applications like smart homes, smart cities, intelligent transportation, industrial automation, precision agriculture, environment monitoring, and healthcare.

The extensive use of IoT devices has brought about a new means of producing and using information, making processes efficient and cost-effective and leading to intelligent decision-making. While computing systems are composed of computers, IoT systems involve an overwhelmingly large number of communicating devices through various communication channels. These devices produce large amounts of data that can be analyzed for automated and predictive decision-making. However, most IoT devices have been developed with little or no computing power, memory space, and energy. The implication is that implementing security measures may be difficult in such devices, meaning that security is neglected in the process of device development.

Besides, due to the fast expansion of the Internet of Things (IoT), the rate and complexity of cyberattacks have increased significantly. Cybercrimes including DDoS attacks, brute force attacks, reconnaissance attacks, spoofing, botnet attacks, and manipulation of data have become more frequent in the IoT environment. IoT devices, which may have been compromised, could be used to disrupt infrastructures, obtain confidential information, invade user privacy, and carry out attacks on large scale. The Mirai botnet attack is a high-profile example which indicates that how vulnerable IoT devices could be leveraged to compromise millions of internet devices.

In order to protect IoT networks from various emerging threats to their cybersecurity, Intrusion Detection Systems (IDSs) are becoming a vital element of contemporary network security. IDS is a tool that constantly monitors all network traffic and activities in order to detect any malicious activity and intrusions in the network system. Traditional IDS tools basically use the signature-based detection approach, wherein network traffic is checked against a database of attack signatures that have already been identified. These systems prove to be highly effective in detecting known attacks, but tend to fail when it comes to detecting zero-day attacks, evolving malware, and new types of intrusions.

The recent improvements made in the areas of Artificial Intelligence (AI), Machine Learning (ML) and Deep Learning (DL) have greatly improved the functionality of the intrusion detection system. Different from the traditional IDS, AI-enabled IDSs can automatically learn from historical network traffic patterns and detect attacks both known and unknown by improving their accuracy. ML algorithms can successfully classify network traffic based on hand-crafted feature sets, whereas DL algorithms automatically extract the high-level representations of the network traffic patterns from the large dataset. This intelligent approach not only achieves better performance in detecting cybersecurity threats but also reduces the rate of false alarms.

Despite all these developments, creating an intrusion detection system using artificial intelligence in IoT networks is still a difficult problem to solve. Datasets of IoT usually consist of redundant features, irrelevant, and highly correlated features, which add to the complexity of computation and can harm classification accuracy. Moreover, class imbalance, noise in the data, and the high dimensionality of the traffic network are some of the problems faced during the training of the models. Hence, preprocessing and selecting features and evaluating the model becomes extremely important.

In view of these challenges, this project focuses on the development of an AI-based intrusion detection framework for IoT networks by integrating appropriate data preprocessing techniques, feature selection methods, and machine learning and deep learning algorithms. The objective is to enhance intrusion detection performance while reducing computational overhead, thereby contributing to the development of intelligent and reliable security solutions for modern IoT environments.

1.2. PROBLEM STATEMENT

Due to the quick growth in IoT, there has been an increase in the number of devices that interact and operate in different environments, such as smart homes, healthcare systems, industrial automation, smart cities, and intelligent transport systems. Despite the fact that these devices offer better automation and monitoring, they bring about many security issues because of their poor computing capacity, little memory, and light communication protocol. These IoT devices lack good security settings, which makes them vulnerable to attacks from cyber criminals.

The heterogeneity and distribution of the IoT networks make it even harder to implement any security mechanism. The IoT devices exchange private information in numerous communication layers that include perception layer, network layer, and application layer. All these layers are subject to attacks that can be categorized differently depending on the layer. For instance, the perception layer is prone to physical attacks, sensor attack, node capture, and jamming attacks. The network layer becomes a target for Man-in-the-Middle (MITM) attack, spoofing, replay attack, routing attacks, and network reconnaissance attack. In the application layer, the attacker uses software flaws via DDoS attack, malware, unauthorized access, and service disruption.

For overcoming such issues, intrusion detection systems (IDSs) are used extensively for monitoring network traffic and detecting malicious activities. But there are certain limitations associated with traditional methods of intrusion detection systems, namely, signature-based detection system and rule-based detection systems, which are used in dynamic IoT environment. Signature-based IDS uses predefined attack signature for detecting malicious activities. Even though it is highly accurate in detecting attacks that have already been known to us, but it cannot detect new kinds of zero-day attacks because their signature does not exist in the database.

In like manner, rule-based IDS relies on handcrafted rules and expert knowledge in order to recognize malicious behaviors. The process of creating and updating the rule is time-consuming and cumbersome, especially as new attack patterns are constantly emerging. There is need for constant updates in order to maintain good performance of the system, which makes the use of rule-based methods not very scalable in IoT

environments. In addition, both approaches face challenges in dealing with the massive data produced by the IoT networks.

A second problem faced while designing intelligent intrusion detection models is that of high-dimensional data sets, which are filled with redundant and irrelevant features along with high correlations between different features. High-dimensional data sets add to computational overheads, increase the time required for model training, and adversely affect the detection performance because of noise and irrelevant features in the dataset.

Therefore, there is a need for an intelligent, data-driven intrusion detection framework capable of efficiently analyzing IoT network traffic, selecting the most relevant features, and accurately distinguishing between normal and malicious activities. Such a framework should provide high detection performance while minimizing computational overhead, making it suitable for deployment in modern IoT environments.

1.3. MOTIVATION

The emergence of IoT technology at a breakneck pace has changed how different devices interact and communicate with each other through various applications. The smart home, healthcare system, automation industry, agriculture, and transportation all have become dependent on IoT devices to provide automation and real-time monitoring. However, the use of IoT devices has increased the number of attack surfaces and made the network vulnerable to attacks by the cyber-criminals. It has become important to maintain the integrity of these networks.

The conventional methodologies for intrusion detection include signature based and rule based systems. Even though these methods are quite efficient in detecting any attacks that have already been witnessed before, they lack the capability of detecting new attacks or any zero-day attacks since they depend on pre-programmed signatures and detection rules. In addition, the constant changing nature of cyber attacks necessitates regular updates of these detection systems.

The increasing volume of network traffic generated by IoT devices presents another major challenge for intrusion detection systems. Modern IoT datasets contain a large number of features, many of which may be redundant, irrelevant, or highly correlated.

Training machine learning and deep learning models using all available features increases computational complexity, prolongs training time, and may reduce the overall detection performance due to the inclusion of noisy information. Therefore, selecting the most informative features before model training is essential for improving both efficiency and classification accuracy.

The recent developments in the field of Artificial Intelligence (AI) such as Machine Learning (ML) and Deep Learning (DL) technologies have been proven very effective in improving the intrusion detection techniques. As opposed to traditional systems, these smart systems can easily learn complicated features through analysis of the network traffic itself. The machine learning algorithms make use of an effective feature set selection and classification, whereas the deep learning algorithms can easily learn the hierarchy in the network traffic.

Driven by the above challenges and opportunities, this study intends to propose an intelligent intrusion detection approach for IoT network by employing appropriate data preprocessing, feature selection, and AI approaches for classification. In this regard, the CICIoT2023 dataset is used in this study where data preprocessing and feature selection using Pearson Correlation Coefficient (PCC) are done in order to select the most significant features, and RF, ANN, and CNN are evaluated as classifiers for intrusion detection task. Comparison of these models based on performance measures will help to find out the efficient model for intrusion detection.

1.4. PROPOSED SOLUTION

To address the security challenges associated with Internet of Things (IoT) networks, this project implements an Artificial Intelligence (AI)-based intrusion detection system by following the methodology proposed in the selected research study. The implementation focuses on reproducing the data preprocessing, feature selection, model development, and evaluation pipeline presented in the reference work, while analyzing its effectiveness on the selected IoT intrusion dataset.

The proposed methodology begins with the CICIoT2023 dataset, which contains network traffic generated from both normal and malicious IoT activities. Since real-world datasets often contain duplicate records, missing values, infinite values, and redundant information, appropriate preprocessing techniques are applied to improve data quality before model development. The dataset is cleaned and prepared to ensure that it is suitable for training and evaluating machine learning and deep learning models.

Following data preprocessing, Exploratory Data Analysis (EDA) is performed to understand the characteristics of the dataset, identify feature distributions, examine class imbalance, and gain insights into the relationship between different network traffic attributes. This analysis assists in selecting an appropriate preprocessing and feature selection strategy.

To reduce the dimensionality of the dataset and eliminate redundant features, the methodology employs the Pearson Correlation Coefficient (PCC) feature selection technique, as described in the reference paper. PCC measures the correlation between individual features and the target class, enabling the selection of the most relevant attributes for intrusion detection. By retaining only informative features, the computational complexity of the learning models can be reduced while preserving classification performance.

Once the data processing stage of feature selection has been completed, the processed data set is split into training and testing sets. Then, the features selected are used to create and analyze three classification models that include the Random Forest (RF), Artificial Neural Network (ANN), and Convolutional Neural Network (CNN). They are all trained with the processed data set to provide an even ground of comparison.

The overall objective of this implementation is to reproduce the methodology presented in the selected research work and evaluate the performance of traditional machine learning and deep learning techniques for IoT intrusion detection. The outcomes of this study provide insights into the effectiveness of the implemented models and establish a foundation for future enhancements and comparative analyses.

2. OBJECTIVES

- To collect and preprocess an IoT network traffic dataset containing both normal and malicious traffic, including different attack types for binary and multi-class classification.
- To develop and compare Machine Learning and Deep Learning models for accurately detecting malicious IoT network traffic.
- To evaluate the developed intrusion detection models using metrics such as Accuracy, Precision, Recall, F1-Score, ROC-AUC, and False Alarm Rate and develop a visual dashboard to monitor and present their performance.

3. LITERATURE SURVEY

3.1. INTRODUCTION

Fast development of the Internet of Things has resulted in an increasing requirement for effective cybersecurity tools that can ensure the security of the devices connected in the network from possible cyber attacks. With the expansion of IoT networks in different domains of applications, various techniques of intrusion detection have been suggested in order to detect any possible malicious behavior in the network while maintaining the required efficiency and reliability of IoT. Within the last ten years, intrusion detection systems have become more advanced from simple signature-based methods to machine learning-based systems.

A number of researchers have proposed the use of machine learning techniques like Decision Tree, Random Forest, Support Vector Machine, Naive Bayes, and Ensemble Learning in detecting attacks in a network. These approaches have shown encouraging results in recognizing attack signatures and possess better adaptability than the rule-based approaches used traditionally. Nevertheless, most of the classical machine learning approaches depend on hand-crafted feature engineering and their performance may drop while processing high-dimensional data sets which include redundant features.

Given the growing availability of large cybersecurity datasets, deep learning methods have been gaining popularity because of their potential to learn advanced feature representations autonomously from network traffic data. Some of the machine learning algorithms that have been used for the purpose of intrusion detection include ANN, CNN, RNN, and LSTM. These methods have been successful in increasing the detection accuracy for complex attack situations. However, despite these developments, there are issues such as computational complexity, feature redundancy, class imbalance, and scalability that affect intrusion detection system performance.

To address these challenges, recent research has focused on combining efficient data preprocessing techniques with feature selection methods and intelligent classification algorithms. Feature selection techniques such as the Pearson Correlation Coefficient (PCC) have been widely employed to eliminate redundant features, reduce computational complexity, and improve model performance. The research paper

selected for this project follows this approach by integrating PCC-based feature selection with machine learning and deep learning models for IoT intrusion detection. The following sections present a detailed review of the existing literature, highlight recent developments in AI-based intrusion detection systems, and identify the research work that forms the foundation of the present implementation.

3.2. SUMMARY OF EXISTING RESEARCH PAPERS

There has been an improvement in technology of IoT in recent times, resulting in increased research efforts on securing devices that are connected against potential threats on the Internet. As shown by the studies in this paper, the development of the IoT in fields such as smart homes, health care, automation industry, transportation, and smart cities has posed a challenge for achieving a secure communication between resource constrained devices. This is because traditional methods of security cannot guarantee protection for modern day IoT environments owing to the heterogeneity of network traffic. As a result, there has been increased interest in creating IDSs that are intelligent enough to detect any threat with minimal computational overhead.

As far as the literature shows, intrusion detection techniques have come a long way during the last ten years. The first IDS technologies used signature- and rule-based approaches for traffic classification, where attack signatures and rules, which were pre-defined by experts, were used to detect attacks. This approach proved to be quite efficient in case of known attacks; however, it lacked adaptability towards zero-day attacks and new intrusion patterns. Besides, the constant update of signature and rule databases makes these approaches ineffective in ever-changing IoT environments.

The Machine Learning (ML) approach has proved to be one of the most common methods for IoT intrusion detection. There have been multiple studies focused on the evaluation of different supervised machine learning techniques, such as Decision Trees, Random Forest, SVM, Naive Bayes, K-Nearest Neighbors, Logistic Regression, and also ensemble learning approaches. Based on the analyzed literature, Random Forest has been one of the most popular classifiers used in the studies due to its efficiency and reduced risks of overfitting. Moreover, all kinds of ensemble learning approaches have shown better classification results than other approaches, as well as low computational complexity.

Due to increased access to cybersecurity big data, there is a lot of focus on DL approaches owing to their potential in automatic hierarchical representation learning. In the reviewed papers, many different types of ANN models such as CNNs, RNNs, LSTMs, and hybrid deep learning architectures have been used for intrusion detection. Among all these approaches, CNNs have shown better results since they are effective in capturing complex patterns in network traffic without relying much on manual feature extraction. However, one of the limitations of deep learning approaches is that they need bigger datasets, more computation resources, and longer training time as compared to traditional ML models.

Another significant finding from the reviewed literature is the increasing importance of data preprocessing and feature selection. Almost all publicly available intrusion detection datasets include redundant, irrelevant, or highly correlated features which might adversely impact the performance of models and make the computation more expensive. For this reason, many different feature selection algorithms have been investigated, such as filter methods, wrapper methods, embedded methods, and hybrid methods. One of the filter algorithms is the Pearson Correlation Coefficient (PCC), which has attracted much attention because of its low computational cost and effectiveness in selecting highly relevant features while filtering out redundant features.

Benchmark datasets have been pointed out by the literature as a necessary factor to test the intrusion detection systems. These are some of the widely used benchmark datasets – NSL-KDD, UNSW-NB15, CICIDS2017, BoT-IoT, IoTID20, and in recent years CICIoT2023. The datasets have various attack cases as well as traffic features for evaluation of the performance of the research models in various network environments. Evaluations have been carried out through parameters such as Accuracy, Precision, Recall, F1-Score, ROC curve, AUC, and Confusion Matrix.

Even with considerable advances in the state of art as described above, there are many issues that continue to hamper the practical application of AI-driven IDS in IoT environments. Some of the problems that have been identified include issues related to high-dimensional data, class imbalance, noisy network traffic, complexity of algorithms, low interpretability of models, and constant introduction of novel attacks. Increasing number of works deal with creating lightweight and scalable solutions to

intrusion detection problem that provide high precision through efficient preprocessing, feature selection, and machine learning models.

In general, the discussed literature illustrates an evident shift from traditional intrusion detection techniques to intelligent AI-based systems. The overall insights indicate that the combination of successful data preprocessing, efficient feature selection, and powerful machine/deep learning approaches can be considered an effective way towards enhancing the performance of intrusion detection systems within contemporary IoT environments. The outlined findings serve as the base for the methodology used within the current research, as suggested in the chosen PCC-CNN literature review paper.

3.3. SELECTED RESEARCH PAPER

The implementation used in this work is inspired from the research paper titled "Anomaly Based Intrusion Detection System for IoT Application" authored by Mansi Bhavsar, Kaushik Roy, John Kelly, and Odeyomi Olusola. In the research paper, an AI-based intrusion detection system has been suggested, which uses the Pearson Correlation Coefficient (PCC) method to select features along with a CNN for the purpose of anomaly detection in IoT networks. The aim of the research is to increase the performance of intrusion detection by removing redundant features before classification.

The authors recognize that IoT devices continuously generate large volumes of heterogeneous network traffic, making traditional intrusion detection techniques less effective due to increasing computational complexity and evolving attack patterns. They emphasize that datasets used for intrusion detection often contain redundant and highly correlated features, which unnecessarily increase training time and may reduce the effectiveness of learning algorithms. To address this challenge, the proposed framework first performs feature selection using the Pearson Correlation Coefficient (PCC), allowing only the most relevant features to be used for model training.

The methodology that will be adopted for this study involves various stages that are carried out sequentially, starting with data pre-processing. Firstly, the datasets are processed to ensure that they are ready for use in the modeling process. In the next stage, feature selection is performed using PCC. The correlation value of each feature and the target variable is determined. The features having higher correlation values are

selected while those having lower values are disregarded since they do not add much information to the classifier.

Following feature selection, the selected features are provided as input to a Convolutional Neural Network (CNN). Although CNNs are traditionally associated with image processing, the authors demonstrate that one-dimensional convolutional operations can effectively learn local feature relationships present in network traffic data. The CNN automatically extracts hierarchical representations from the selected features, enabling the model to distinguish between normal and malicious network traffic with high accuracy. The learning capability of CNN also reduces the dependence on manually engineered features, making it suitable for complex intrusion detection tasks.

In order to conduct performance analysis of the suggested framework, three datasets that are widely used for intrusions detections were used: NSL-KDD, CICIDS2017, and IoTID20. These datasets together cover different classes of cyber attacks as well as different networks, which allows the methodology to be tested under varied operational conditions. Evaluation of performance was done via calculation of such traditional measures of classification as accuracy, precision, recall, and f1-score.

The experimental results provided by the paper suggest that using the combination of PCC-based feature selection with the CNN model increases intrusion detection accuracy while decreasing the computational complexity of the process. This is achieved because of the reduction of unnecessary data prior to the training process which decreases the amount of calculations needed for learning. Furthermore, the results confirm that CNN can detect complicated patterns in the IoT network traffic effectively.

One of the major strengths of the proposed research is its systematic combination of efficient feature selection and deep learning-based classification. Instead of directly training a deep learning model using all available features, the authors first reduce the dimensionality of the data through PCC-based filtering, thereby improving both computational efficiency and model performance. Additionally, evaluating the framework on three different benchmark datasets enhances the credibility of the reported results by demonstrating consistent performance across multiple network environments.

The study, however, also presents certain limitations. The experimental evaluation is restricted to the NSL-KDD, CICIDS2017, and IoTID20 datasets, which may not fully represent the latest attack patterns observed in modern IoT environments. Recent datasets such as CICIoT2023, which include newer attack scenarios and more diverse IoT traffic, were not considered during evaluation. Furthermore, the research primarily focuses on the CNN-based classification model and does not provide a detailed comparative analysis with multiple machine learning and deep learning classifiers under identical preprocessing conditions. These observations provide opportunities for further evaluation using updated datasets and additional classification models.

The methodology proposed in this research paper serves as the foundation for the implementation carried out in the present project. While preserving the core preprocessing and feature selection strategy proposed by the authors, the current work evaluates the methodology using the CICIoT2023 dataset and investigates the performance of Random Forest (RF), Artificial Neural Network (ANN), and Convolutional Neural Network (CNN) models under a common experimental framework. This implementation facilitates a broader comparative analysis while maintaining consistency with the methodology presented in the selected research paper.

The selected research paper was chosen because it presents a structured and well-validated methodology that integrates feature selection with deep learning for IoT intrusion detection. Its evaluation on multiple benchmark datasets, together with its clear preprocessing pipeline, makes it an appropriate foundation for implementation and comparative analysis. The methodology is reproducible and provides a suitable basis for assessing its performance on the CICIoT2023 dataset using additional classification models.

3.4. RESEARCH GAP

Despite the significant progress achieved in Artificial Intelligence (AI)-based Intrusion Detection Systems (IDSs), the literature survey indicates that several challenges continue to exist in securing Internet of Things (IoT) environments. Researchers have successfully demonstrated the effectiveness of machine learning and deep learning techniques in detecting malicious network traffic; however, factors such as rapidly evolving cyberattacks, increasing network complexity, and the growing volume of IoT-generated data continue to influence the performance and adaptability of intrusion detection systems.

A common observation from the reviewed literature is that many intrusion detection models have been evaluated using benchmark datasets such as NSL-KDD, CICIDS2017, and IoTID20. Although these datasets have significantly contributed to the advancement of intrusion detection research, they may not fully represent the characteristics of modern IoT environments or recently emerging attack patterns. As IoT technology continues to evolve, newer datasets containing more diverse network traffic and attack scenarios have become available, creating opportunities to evaluate existing methodologies under updated conditions.

Another important observation is that several studies primarily focus on evaluating a single classification model or comparing only a limited number of machine learning algorithms. While these investigations demonstrate the effectiveness of individual approaches, a broader comparative analysis involving both traditional machine learning and deep learning techniques under a common preprocessing framework can provide a better understanding of their relative strengths and limitations.

The selected research paper proposes an effective framework that combines Pearson Correlation Coefficient (PCC) based feature selection with a Convolutional Neural Network (CNN) for anomaly detection in IoT networks. The reported results demonstrate that reducing redundant features before classification improves computational efficiency while maintaining high detection accuracy. However, the evaluation presented in the paper is limited to the benchmark datasets considered by the authors and primarily focuses on the CNN classifier.

Motivated by these observations, the present project implements the methodology proposed in the selected research paper and evaluates its applicability using the

CICIoT2023 dataset, which contains recent IoT network traffic and attack scenarios. In addition to implementing the CNN model, the project also evaluates the performance of Random Forest (RF) and Artificial Neural Network (ANN) models using the same preprocessing and feature selection pipeline. This enables a consistent comparison of machine learning and deep learning approaches while preserving the core methodology proposed in the reference study.

The objective of the present work is not to propose a new intrusion detection framework but to reproduce, implement, and evaluate the selected methodology using a modern benchmark dataset and multiple classification models. The findings obtained from this implementation are expected to provide insights into the effectiveness, adaptability, and practical applicability of the proposed methodology in contemporary IoT intrusion detection scenarios.

4. SPECIFICATIONS

4.1. HARDWARE SPECIFICATIONS

The implementation of the proposed intrusion detection system was carried out using a high-performance Linux-based GPU server to efficiently process the large-scale CICIoT2023 dataset and train machine learning and deep learning models. The computational resources available on the server significantly reduced preprocessing and model training time compared to conventional desktop environments.

Component	Specification
Processor	Multi-core CPU
Operating System	Linux
GPU	NVIDIA GPU with CUDA support
RAM	High-capacity system memory
Storage	SSD Storage
Internet	High-speed network connectivity

Table 4.1. Hardware Specifications

4.2. SOFTWARE SPECIFICATIONS

The project was developed using Python and various open-source libraries for data preprocessing, visualization, feature selection, machine learning, and deep learning. The Linux operating system provided a stable environment for executing the preprocessing pipeline and training the classification models.

Software	Purpose
Ubuntu/Linux	Development Environment
Python 3.x	Programming Language
Jupyter Notebook / VS Code	Development Environment
Google Colab (<i>Initial Stage</i>)	Preliminary experimentation
Linux GPU Server	Final implementation
Git	Version Control

Table 4.2. Software Specifications

4.3. PROGRAMMING LANGUAGES AND LIBRARIES

The implementation of the proposed intrusion detection system utilized several Python libraries to perform data preprocessing, feature engineering, model development, visualization, and performance evaluation.

Library	Purpose
Pandas	Data manipulation
NumPy	Numerical computations
Matplotlib	Data visualization
Scikit-learn	ML algorithms and evaluation
TensorFlow / Keras	ANN and CNN implementation
Joblib	Saving trained models

Table 4.3. Programming Languages and Libraries

4.4. DATASET SPECIFICATION

The present study utilizes the CICIoT2023 dataset for implementing and evaluating the proposed intrusion detection framework. The dataset is one of the latest publicly available benchmark datasets designed specifically for Internet of Things (IoT) intrusion detection research. It contains both normal and malicious network traffic generated from realistic IoT environments and includes a wide range of modern cyberattack scenarios.

The dataset is made up of several CSV files with network traffic features that show various communication traits and attack types. The data was preprocessed before developing the models through combining several files, discarding duplicate records, dealing with missing and infinite values, feature selection using the Pearson Correlation Coefficient (PCC), and normalization of the data.

As opposed to earlier benchmark datasets like NSL-KDD, CICIDS2017, and IoTID20, the CICIoT2023 dataset has more current attack patterns and also traffic in large volume and therefore it is ideal for assessing the performance of modern AI-based IDSs.

4.5. PERFORMANCE EVALUATION METRICS

The performance of the implemented classification models is evaluated using standard classification metrics widely adopted in intrusion detection research. These metrics provide a comprehensive assessment of the model's capability to correctly classify normal and malicious network traffic.

The primary evaluation metrics include:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix

These performance measures are calculated after testing each classification model and are used to compare the effectiveness of the Random Forest (RF), Artificial Neural Network (ANN), and Convolutional Neural Network (CNN) models.

5. ARCHITECTURE

5.1. SYSTEM OVERVIEW

The proposed system implements an Artificial Intelligence (AI)-based Intrusion Detection System (IDS) for Internet of Things (IoT) environments by following the methodology presented in the selected research paper. The objective of the system is to accurately distinguish between normal and malicious network traffic through a structured pipeline consisting of data preprocessing, feature selection, model training, and performance evaluation.

The implementation begins with the CICIoT2023 dataset, which contains network traffic generated from IoT devices under both normal operation and multiple cyberattack scenarios. Since the raw dataset contains duplicate records, missing values, and redundant features, preprocessing techniques are applied to improve data quality before model development.

Following preprocessing, Exploratory Data Analysis (EDA) is performed to understand the characteristics of the dataset and identify important statistical properties. The processed dataset then undergoes Pearson Correlation Coefficient (PCC) based feature selection to eliminate redundant attributes and retain only the most relevant features required for intrusion detection.

This selected feature set is then employed to train and test three types of classification algorithms: Random Forest (RF), Artificial Neural Network (ANN), and Convolutional Neural Network (CNN). Evaluation of these classifiers is performed by using various metrics such as Accuracy, Precision, Recall, F1-Score, and Confusion Matrix. The complete process helps in comparing machine learning and deep learning approaches under the same preprocessing technique.

5.2. SYSTEM ARCHITECTURE

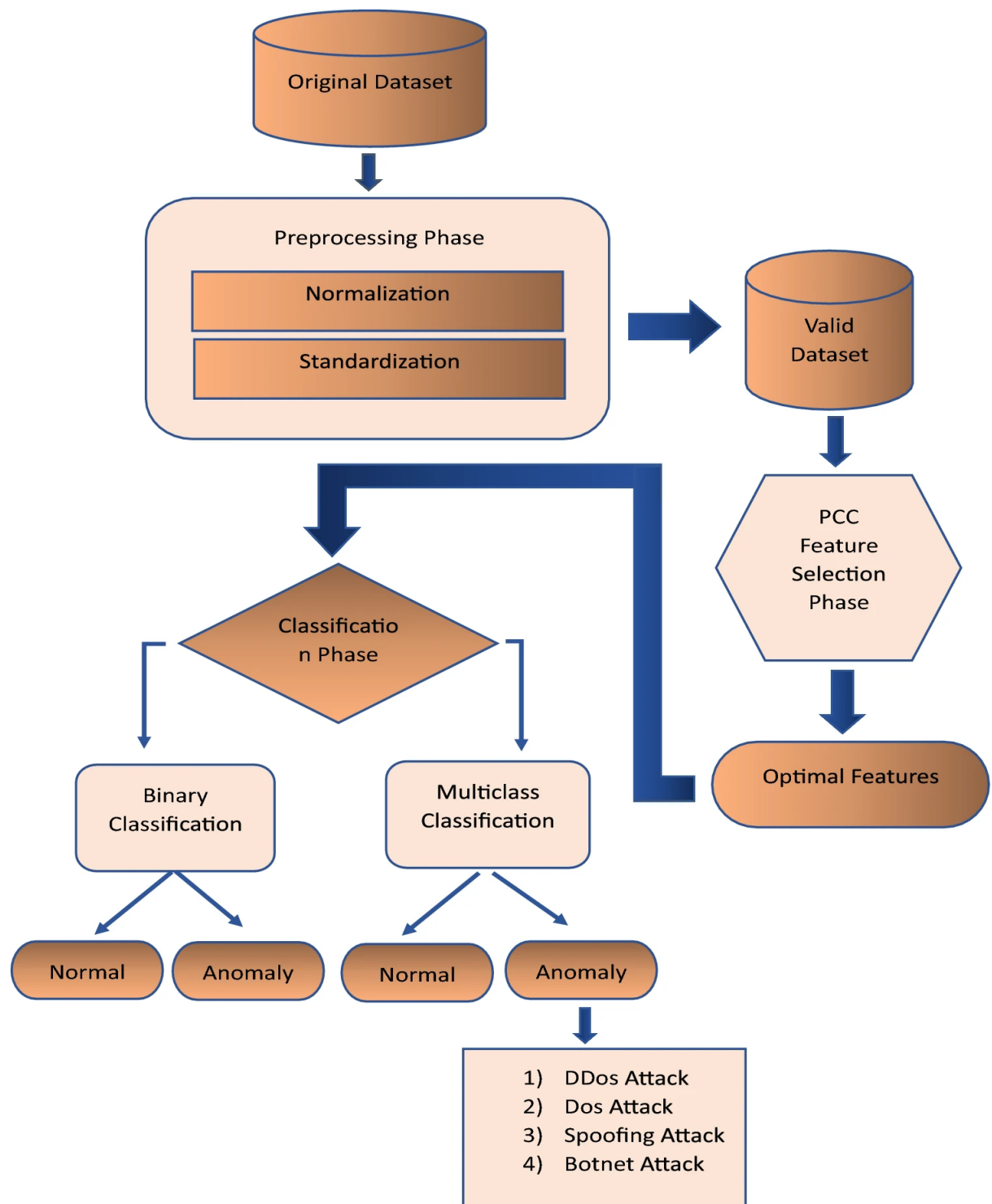


Figure 5.1. Architecture Diagram

The architecture adopted in the present work is based on the framework proposed in the selected research paper, "Anomaly-Based Intrusion Detection System for IoT Application." The framework provides a systematic approach for detecting anomalies in IoT network traffic by integrating data preprocessing, feature selection, deep learning-based classification, and performance evaluation. The overall architecture consists of multiple interconnected stages, where the output of each stage serves as the input for the subsequent stage, thereby ensuring a structured and efficient intrusion detection process.

The implementation begins with the acquisition of network traffic datasets containing both normal and malicious communication records. In the original research paper, the proposed framework was evaluated using the NSL-KDD, CICIDS2017, and IoTID20 benchmark datasets to validate its effectiveness across different intrusion detection scenarios. These datasets contain various categories of cyberattacks together with legitimate network traffic, providing a comprehensive environment for evaluating the proposed methodology.

After acquiring the datasets, there is a process referred to as preprocessing of the raw network traffic. Preprocessing entails the preparation of data ready for use in machine learning and deep learning tasks through getting rid of duplicates, managing missing and inconsistent values, and cleaning the dataset in such a way that it becomes fit for use in other processes.

Following preprocessing, the framework applies Pearson Correlation Coefficient (PCC) based feature selection. The purpose of this stage is to measure the relationship between each feature and the target variable while identifying redundant or weakly correlated attributes. Features demonstrating higher relevance to intrusion detection are retained, whereas less significant features are discarded. This process reduces dataset dimensionality, decreases computational complexity, and improves the efficiency of the classification model without sacrificing important information.

The chosen characteristics are then fed to a Convolutional Neural Network (CNN) classifier. The CNN is able to extract meaningful patterns from the pre-processed network traffic data, and learn hierarchical representation of the features that can classify normal from abnormal activities. Using multiple convolutional and learning steps, the CNN is able to understand the relationships existing within the data.

The final stage of the architecture focuses on evaluating the performance of the trained classification model using standard performance metrics such as Accuracy, Precision, Recall, F1-Score, and the Confusion Matrix. These metrics provide a comprehensive assessment of the model's capability to correctly classify network traffic and measure its effectiveness in detecting cyberattacks while minimizing false classifications.

Although the present project follows the overall architecture proposed in the selected research paper, the experimental implementation is performed using the CICIOT2023 dataset. Furthermore, in addition to reproducing the CNN-based methodology, the implementation also evaluates Random Forest (RF) and Artificial Neural Network (ANN) models using the same preprocessing and feature selection pipeline. This enables a comparative analysis of machine learning and deep learning approaches while preserving the fundamental architecture proposed by the original study.

5.3. SYSTEM MODULES

This architecture is divided into several functional modules which are responsible for providing the desired functionality of detecting any potential intrusion. Each module has its own particular role in the whole process, starting from the data collection phase up until the evaluation of the trained classifiers. This design makes the whole process much clearer and guarantees that each step adds to the performance of the intrusion detection system.

5.3.1. Dataset Collection

The first module of the proposed system is responsible for acquiring the network traffic dataset required for intrusion detection. In the original research paper, the methodology was evaluated using the NSL-KDD, CICIDS2017, and IoTID20 benchmark datasets to validate the effectiveness of the proposed framework across different intrusion detection scenarios. These datasets contain both legitimate and malicious network traffic representing multiple categories of cyberattacks.

For the implementation carried out in the present project, the **CICIOT2023** dataset is utilized as the primary source of network traffic. The dataset contains realistic IoT communication generated from multiple smart devices together with a wide range of contemporary cyberattacks. Its large size and diverse attack categories make it suitable for evaluating modern Artificial Intelligence-based intrusion detection systems.

The collected dataset serves as the input for all subsequent stages of the implementation, including preprocessing, feature selection, model training, and performance evaluation.

5.3.2. Data Preprocessing

The process of data preprocessing is one of the most important steps in the intrusion detection system, since the quality of the input data affects the efficiency of the classification models. The network traffic dataset can include duplicate rows, missing and infinite values, inconsistency and redundancy in the data, which can adversely affect the classification process.

The process of preprocessing includes the cleaning of the dataset from duplicate rows, missing and infinite values, as well as data consistency. After preprocessing, the dataset is prepared for the use in machine learning and deep learning models. Data preprocessing is necessary not only for increasing the data reliability but also for decreasing computational complexity.

5.3.3. Exploratory Data Analysis (EDA)

After preprocessing, Exploratory Data Analysis (EDA) is performed to understand the characteristics of the dataset before model development. EDA provides valuable insights into feature distributions, class imbalance, statistical properties, and relationships among different network traffic attributes.

Various visualization techniques and statistical analyses are employed to examine the quality of the dataset and identify potential anomalies or inconsistencies. The observations obtained during this stage assist in validating the preprocessing process and provide a better understanding of the data before feature selection and model training.

5.3.4. Feature Selection using Pearson Correlation Coefficient (PCC)

Feature selection is an essential stage in the proposed intrusion detection framework because high-dimensional datasets often contain redundant, irrelevant, or highly correlated features that increase computational complexity without significantly improving classification performance. The presence of unnecessary attributes may also introduce noise into the learning process, resulting in reduced model accuracy and longer training times.

To address this issue, the present work follows the feature selection methodology proposed in the selected research paper by employing the Pearson Correlation Coefficient (PCC). PCC is a statistical technique used to measure the strength and direction of the linear relationship between each feature and the target variable. Features exhibiting stronger correlations with the target class are considered more informative for intrusion detection, whereas features with weak or redundant relationships can be eliminated.

After preprocessing and exploratory data analysis, the correlation values of all selected features are calculated. Based on the obtained correlation coefficients, only the most relevant attributes are retained for model development. This process reduces the dimensionality of the dataset while preserving the information necessary for distinguishing between normal and malicious network traffic.

The application of PCC-based feature selection provides several advantages. It reduces the computational resources required during model training, minimizes the effect of redundant information, improves training efficiency, and enhances the overall performance of both machine learning and deep learning models. The reduced feature set obtained through PCC serves as the input for all classification models implemented in this project.

5.3.5. Random Forest (RF) Module

Random Forest algorithm is one of those machine learning algorithms that have been used in this research for detecting intrusion. Random Forest is an ensemble learning method that involves building a number of decision trees during the training process and combining the outcome of each tree to derive the final classification.

In this project, the Random Forest model is trained using the feature set obtained after PCC-based feature selection. During training, each decision tree learns different characteristics of the network traffic using randomly selected subsets of the training data and features. The final prediction is determined through majority voting among all decision trees.

The Random Forest model serves as a baseline machine learning classifier in this study. Its performance is later compared with Artificial Neural Network (ANN) and Convolutional Neural Network (CNN) models to evaluate the effectiveness of traditional machine learning techniques relative to deep learning approaches for IoT intrusion detection.

5.3.6. Artificial Neural Network (ANN) Module

Artificial Neural Networks (ANNs) are widely used in classification problems due to their ability to model complex and non-linear relationships between input features and output classes. Inspired by the structure of the human brain, ANNs consist of interconnected neurons organized into input, hidden, and output layers.

In the proposed implementation, the ANN model receives the selected features generated through PCC-based feature selection. During training, the network learns the underlying patterns of normal and malicious network traffic by adjusting the weights of interconnected neurons using backpropagation and optimization algorithms.

The ANN model provides a deep learning-based approach for intrusion detection and enables the project to compare shallow ensemble learning methods with neural network-based classification. The experimental results obtained from the ANN are evaluated using standard performance metrics and compared with the Random Forest and CNN models.

5.3.7. Convolutional Neural Network (CNN) Module

The Convolutional Neural Network (CNN) becomes the main deep learning approach taken from the chosen research article. Even though CNNs have been mostly developed to handle images, the process of applying 1D convolutions allows extracting useful features from structured network traffic information.

The CNN architecture takes input in the form of the optimal feature set selected using the PCC technique and is capable of learning hierarchical representations that can classify whether the traffic is normal or malicious. The use of multiple convolutional layers, activations, and pooling helps the network learn both local and global relationships from the input data.

The CNN implementation represents the core methodology proposed in the selected research paper. In the present work, its performance is analyzed alongside Random Forest and Artificial Neural Network models to determine the effectiveness of different classification approaches under the same preprocessing framework.

5.3.8. Performance Evaluation

The final module of the proposed system focuses on evaluating the performance of the implemented classification models. After training, the Random Forest, Artificial Neural Network, and Convolutional Neural Network models are tested using unseen network traffic data to assess their ability to correctly classify normal and malicious activities.

Performance evaluation is carried out using widely accepted classification metrics, including Accuracy, Precision, Recall, F1-Score, and the Confusion Matrix. These metrics provide a comprehensive assessment of model performance by measuring overall correctness, the ability to identify malicious traffic, the capability to avoid false classifications, and the balance between precision and recall.

The results obtained from each classifier are compared to identify the most effective model for IoT intrusion detection using the CICIoT2023 dataset. The comparative analysis also provides insights into the strengths and limitations of machine learning and deep learning approaches when implemented under a common preprocessing and feature selection framework.

6. WORK DONE

6.1. IMPLEMENTATION WORKFLOW

The implementation of the proposed intrusion detection system was carried out by following a structured workflow beginning with the acquisition of the CICIoT2023 dataset and ending with the comparative evaluation of multiple classification models. Each stage of the workflow contributes to improving the quality of the data and enhancing the effectiveness of the intrusion detection system.

Initially, multiple CSV files from the CICIoT2023 dataset were merged to form a unified dataset suitable for analysis. Since the merged dataset contained millions of records, sampling techniques were employed to obtain a representative subset that could be processed efficiently without affecting the distribution of attack classes.

The sampled dataset then underwent data cleaning to remove duplicate records, handle missing values, eliminate infinite values, and improve overall data quality. Exploratory Data Analysis (EDA) was subsequently performed to understand the characteristics of the dataset and identify important statistical properties before model development.

After the pre-processing stage, the Pearson Correlation Coefficient (PCC)-based feature selection method was used to select the most important features for intrusion detection purposes. After that, the selected features were used to build and test the Random Forest (RF), Artificial Neural Network (ANN), and Convolutional Neural Network (CNN) classification models. The final step was the comparison of the performance of the three classifiers using standard performance evaluation metrics.

6.2. DATASET COLLECTION

The implementation presented in this project utilizes the CICIoT2023 dataset as the primary source of network traffic for intrusion detection. The dataset was selected because it contains recent IoT communication patterns together with a diverse collection of modern cyberattacks, making it suitable for evaluating Artificial Intelligence-based intrusion detection systems.

The CICIoT2023 dataset is distributed as multiple CSV files, where each file represents network traffic generated under different attack scenarios. Collectively, these files contain millions of network traffic records with numerous numerical features

describing communication characteristics and corresponding class labels representing normal and malicious activities.

Due to the large volume of data, it was not practical to directly train the machine learning and deep learning models using all available files. Therefore, the implementation first focused on organizing and preparing the dataset through merging, sampling, preprocessing, and feature selection before model development.

6.3. DATASET MERGING

The CICIoT2023 dataset is provided in the form of several Comma Separated Value (CSV) files, whereby each CSV file consists of network traffic data for various IoT communication scenarios and types of attacks. Because of the existence of separate files for the CICIoT2023 dataset, it becomes necessary to merge these files first before doing any preprocessing and building the models. It makes working with the dataset easier and more consistent.

All the dataset files were sorted into the Linux server environment at the beginning of the implementation process. The Python code for data processing would then be written to automatically load and check the structure of all the CSV files and append their contents to the same dataframe. Prior to concatenation, all the files were made sure to contain the same features in the same order.

After validating the dataset structure, the CSV files were merged into a single consolidated dataset containing all available network traffic records. The merged dataset preserved the original feature representation while integrating data from multiple attack scenarios into one unified dataset. This consolidated dataset served as the foundation for all subsequent preprocessing, exploratory data analysis, feature selection, and classification stages.

Considering the extremely large size of the merged dataset, directly performing machine learning and deep learning experiments on the complete dataset would require substantial computational resources and processing time. Therefore, after successfully merging all available CSV files, the dataset was prepared for the sampling stage, where a representative subset was generated for efficient experimentation while preserving the original class distribution.

6.4. DATASET SAMPLING

The combined CICIoT2023 dataset comprised about 45 million network traffic entries, making the task of doing data pre-processing, feature selection, and training models computationally costly for such a huge volume of data. The computation of such a big volume of data demands much memory, disk space, and processing time, thus complicating the experiment. As a result, a proper sampling technique was used to form a small sample from the original dataset.

In the present work, a 5% stratified sampling technique was employed to create a manageable dataset for experimentation. Stratified sampling was preferred over random sampling because it preserves the original class distribution of the dataset. Since intrusion detection datasets generally contain multiple attack categories with varying numbers of instances, maintaining the proportion of each class is essential to ensure that the sampled dataset accurately represents the original network traffic.

The sampling was done through Python, whereby each category of attacks was separately sampled depending on the required percentage of sampling. This method made sure that the majority and minority classes were adequately represented in the sample set, thus reducing any sampling bias. Consequently, the obtained dataset became fit for further exploratory data analysis and model development.

Following the sampling process, the generated dataset contained approximately 1.47 million records while retaining the original 40 network traffic features. This reduced dataset significantly lowered computational requirements, enabling efficient preprocessing and model training on the available hardware without compromising the diversity of attack patterns present in the original dataset.

The sampled dataset subsequently served as the input for all remaining stages of the implementation, including data cleaning, exploratory data analysis, Pearson Correlation Coefficient (PCC)-based feature selection, and the training of Random Forest (RF), Artificial Neural Network (ANN), and Convolutional Neural Network (CNN) models.

```

✅ Sampling completed successfully!
Sampled dataset saved at:
/content/drive/MyDrive/AI_IDS/DATASETS/MERGED_CSV_Sampled_5Percent.csv

import pandas as pd

sampled = pd.read_csv("/content/drive/MyDrive/AI_IDS/DATASETS/MERGED_CSV_Sampled_5.csv")

print (sampled.shape)

sampled['Label'].value_counts()

(2250933, 40)

```

Figure 6.1. Dataset Sampling and sampled dataset generation

6.5. DATA CLEANING

Cleaning of data is a very vital step in the implementation of machine learning and deep learning algorithms. The quality of data fed to the algorithm determines its performance in terms of accuracy, and therefore it is vital to ensure there are no inconsistencies in the dataset before selecting the features and training the model. Network traffic data in real world contains duplicates, nulls, infinity and wrong values that can influence the learning process.

Once the sampling dataset was created using the CICIoT2023 dataset, the process of data cleaning was done thoroughly. First, the dataset was inspected for any duplicate data entries, which might affect the training process of the model. Duplicate entries were cleaned out to prevent any form of bias in the learning process.

Following duplicate removal, the dataset was analyzed for missing and infinite values. The preprocessing scripts detected a small number of missing values and positive infinite values within a few numerical features. Since these values cannot be directly processed by machine learning and deep learning algorithms, appropriate cleaning operations were performed to replace or remove the affected records. After completing the cleaning process, the dataset was re-evaluated to verify that no missing or infinite values remained.

The cleaning process significantly improved the quality and consistency of the dataset, ensuring that the subsequent stages of exploratory data analysis, feature selection, and model training were performed using reliable and valid network traffic records.

```

Dataset Shape: (2250933, 40)

NUMBER OF DUPLICATES : 776182

NUMBER OF DUPLICATES LEFT : 0

df.to_csv(
    "/content/drive/MyDrive/AI_IDS/DATASETS/MERGED_CSV_SAMPLED_5_DUP_REMOVED.csv",
    index = False
)

df = pd.read_csv("/content/drive/MyDrive/AI_IDS/DATASETS/MERGED_CSV_SAMPLED_5_DUP_REMOVED.csv")

print("Dataset Shape:", df.shape)
print("DUPLICATES :", df.duplicated().sum())

Dataset Shape: (1474751, 40)
DUPLICATES : 0

```

Figure 6.2. Output of Data Cleaning

6.6. EXPLORATORY DATA ANALYSIS (EDA)

After completing the data cleaning process, Exploratory Data Analysis (EDA) was performed to gain a comprehensive understanding of the characteristics of the cleaned dataset before proceeding with feature selection and model development. EDA plays an important role in identifying the statistical properties of the dataset, understanding the distribution of network traffic records, detecting class imbalance, examining feature relationships, and validating the effectiveness of the preprocessing stage.

The cleaned dataset consisted of 1,474,710 network traffic records with 40 network traffic features, representing both normal and malicious network activities. Various statistical analyses and visualization techniques were employed to examine the quality of the dataset and verify that it was suitable for machine learning and deep learning applications.

The primary objectives of the exploratory data analysis were:

- To understand the structure of the dataset.
- To examine the distribution of attack and normal traffic.
- To identify possible class imbalance.
- To analyse the statistical behaviour of numerical features.
- To understand feature relationships before feature selection.

6.6.1. Dataset Overview

The first stage of EDA involved examining the overall structure of the cleaned dataset. The dimensions of the dataset, feature names, data types, and summary statistics were analysed to verify data consistency after preprocessing.

The analysis confirmed that the cleaned dataset contained 1,474,710 records and 40 numerical features, with no remaining missing or infinite values. The dataset was therefore considered suitable for further statistical analysis and model development.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1474751 entries, 0 to 1474750
Data columns (total 40 columns):
```

Figure 6.3. Dataset Information

6.6.2. Statistical Summary

To understand the behaviour of the numerical features, descriptive statistical measures such as mean, standard deviation, minimum value, maximum value, and quartiles were calculated.

These statistics provided an overview of the range and variability of different network traffic features. Since intrusion detection datasets contain features measured on different scales, the statistical summary also helped determine the need for feature scaling during the preprocessing stage.

	count	mean	std	min	25%	50%	75%	max
Header_Length	1474710.0	14.053424	8.799794	0.000000e+00	8.000000	19.880000	20.000000	6.000000e+01
Protocol_Type	1474710.0	10.218475	10.371693	0.000000e+00	6.000000	6.000000	17.000000	4.700000e+01
Time_To_Live	1474710.0	67.850776	17.607711	0.000000e+00	64.000000	64.000000	64.440000	2.550000e+02
Rate	1474710.0	22886.907150	32402.897037	3.973936e-04	6942.947476	16578.276680	30779.364497	7.340032e+06
fin_flag_number	1474710.0	0.075578	0.260457	0.000000e+00	0.000000	0.000000	0.000000	1.000000e+00
syn_flag_number	1474710.0	0.208103	0.397873	0.000000e+00	0.000000	0.000000	0.010000	1.000000e+00
rst_flag_number	1474710.0	0.084742	0.267301	0.000000e+00	0.000000	0.000000	0.000000	1.000000e+00
psh_flag_number	1474710.0	0.095662	0.271660	0.000000e+00	0.000000	0.000000	0.000000	1.000000e+00
ack_flag_number	1474710.0	0.149674	0.328427	0.000000e+00	0.000000	0.000000	0.010000	1.000000e+00
ece_flag_number	1474710.0	0.000042	0.002673	0.000000e+00	0.000000	0.000000	0.000000	1.000000e+00
cwr_flag_number	1474710.0	0.000022	0.001954	0.000000e+00	0.000000	0.000000	0.000000	1.000000e+00
ack_count	1474710.0	10.271491	27.653751	0.000000e+00	0.000000	0.000000	1.000000	1.000000e+02

Figure 6.4. Statistical Summary Output

6.6.3. Class Distribution Analysis

Understanding the distribution of attack and normal traffic is essential before training classification models. Therefore, the class labels were analysed to determine the proportion of malicious and legitimate network traffic present in the dataset.

The analysis revealed that the dataset is highly imbalanced, with malicious traffic representing the majority of the records while normal traffic accounts for only a small proportion. Such imbalance is commonly observed in intrusion detection datasets and must be considered while interpreting classification results.

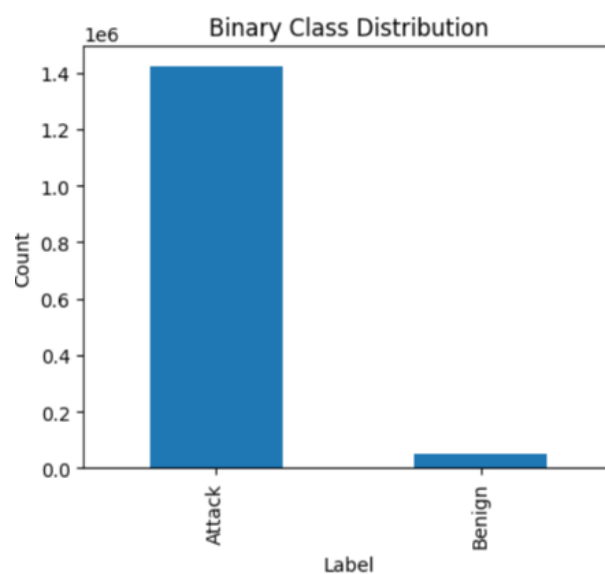


Figure 6.5 Binary Class Distribution

6.6.4. Correlation Analysis

Correlation analysis was performed to understand the relationships among different network traffic features before feature selection. Highly correlated features often contain redundant information and may unnecessarily increase the computational complexity of classification models.

A correlation matrix was generated to visualize the relationships between numerical features. The observations obtained from this analysis supported the subsequent application of the Pearson Correlation Coefficient (PCC) for selecting the most informative features.

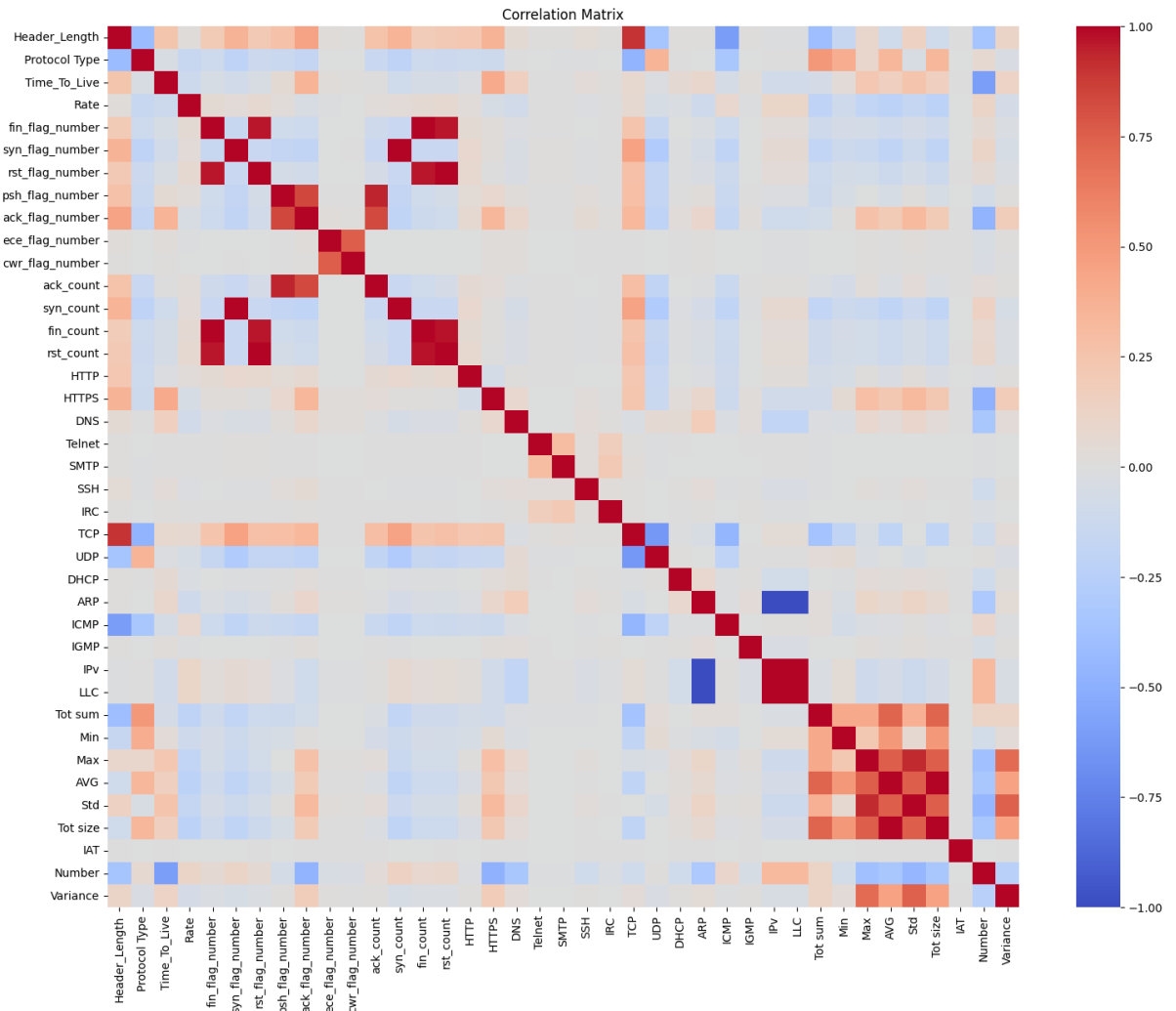


Figure 6.6 Correlation Matrix

6.6.5. Observations from EDA

The exploratory data analysis provided several important observations that influenced the subsequent stages of implementation:

- The cleaned dataset contained no duplicate, missing, or infinite values.
- The dataset consisted of approximately 1.47 million records with 40 network traffic features.
- A significant class imbalance existed between attack and normal traffic.
- Feature values exhibited varying statistical ranges, indicating the importance of appropriate preprocessing.

6.7. BINARY CLASSIFICATION

The original CICIoT2023 dataset contains multiple categories of network traffic representing different cyberattacks together with normal communication records. While multi-class classification enables the identification of specific attack types, the selected research paper focuses on binary intrusion detection, where the primary objective is to determine whether a network traffic instance is normal or malicious.

To maintain consistency with the methodology presented in the selected research paper, the class labels in the CICIoT2023 dataset were converted into a binary representation. During this process, all malicious attack categories were grouped into a single Attack class, while legitimate network traffic was retained as the Normal class. This transformation simplified the classification problem and enabled a direct comparison of the implemented models under a binary intrusion detection framework.

The binary conversion process was implemented using Python by mapping the original labels into two output classes. The Normal class was assigned the label 0, whereas every attack category was assigned the label 1. This approach ensured that the classification models learned to distinguish legitimate network activity from malicious behaviour without considering the individual attack categories.

After completing the binary label conversion, the dataset was re-examined to verify the distribution of both classes. The analysis confirmed that the dataset remained highly imbalanced, with malicious traffic representing the majority of the records. Although class imbalance is a common characteristic of intrusion detection datasets, preserving the original distribution allows the implemented models to be evaluated under realistic network conditions.

The binary dataset generated during this stage served as the final input for the Pearson Correlation Coefficient (PCC) feature selection process and the subsequent training of the Random Forest (RF), Artificial Neural Network (ANN), and Convolutional Neural Network (CNN) models.

Original Label	Converted Label
Normal	0
All Attack Categories	1

Table 6.1. Binary Label Classification

Class	Label	Number of Records
Normal	0	52,519
Attack	1	1,422,191
Total		1,474,710

Table 6.2. Binary Class Distribution

6.8. Pearson Correlation Coefficient (PCC) Feature Selection

Feature selection is an important process in building machine learning or deep learning models especially when dealing with large data sets of intrusion detection with many network traffic features. Redundant features or those that are irrelevant or loosely correlated will only lead to high computational complexity, long training time, and reduced classification accuracy. It is therefore important to choose the most relevant features prior to building the model.

To address this challenge, the present work adopts the Pearson Correlation Coefficient (PCC) feature selection technique proposed in the selected research paper. PCC is a statistical measure that quantifies the strength and direction of the linear relationship between two variables. In this implementation, the correlation between each network traffic feature and the target class was calculated to determine the importance of individual features for intrusion detection.

Correlation coefficient lies in the range of -1 to +1. When the value is close to +1, it shows that there is a high positive correlation, while if the value is close to -1, it depicts high negative correlation. If the value is close to zero, it means there is not much of linear correlation. Characteristics which have higher correlations with the class of interest are important for intrusion detection.

6.8.1. PCC Implementation

Once data pre-processing and binarization of labels had been done, the Pearson Correlation Coefficient was determined for each numerical attribute that appeared in the processed dataset. Each attribute was examined separately in order to find out its importance in differentiating normal and malicious traffic.

Based on the computed correlation values, a predefined threshold was applied to identify the most relevant features. Features with correlation values satisfying the threshold criterion were retained, while weakly correlated attributes were removed from the dataset. This reduced the dimensionality of the dataset and eliminated redundant information before training the classification models.

The implementation followed the same feature selection methodology presented in the selected research paper, thereby ensuring consistency between the proposed framework and the present work.

6.8.2. Feature Ranking

The correlation analysis produced a ranked list of network traffic features according to their relationship with the target class. Features with higher correlation coefficients were considered more significant for intrusion detection. This ranking helped identify the attributes that contributed most to distinguishing between normal and malicious network activities.

The correlation coefficient indicates the strength and direction of the relationship between an individual feature and the target class. A positive correlation suggests that an increase in the feature value is generally associated with the presence of a particular attack class, while a negative correlation indicates an inverse relationship. Features with correlation values close to zero were considered to have a weaker linear relationship with the target variable.

Feature ranking also helps reduce the dimensionality of the dataset by identifying less informative attributes. Removing irrelevant or redundant features can reduce computational complexity, shorten training time, and potentially improve model performance. However, correlation alone does not necessarily indicate that a feature is independently important, since two highly correlated features may contain similar information.

6.8.3. Selected Features

Based on the selected threshold, the final feature subset consisted of the most informative attributes required for intrusion detection. These selected features were used as the input for all classification models implemented in this project.

During the feature selection process, attributes exhibiting very low correlation with the target class were removed. Eliminating these redundant features reduced computational complexity while preserving the information necessary for accurate intrusion detection.

Following PCC feature selection, the dataset dimensionality was reduced from 40 original features to 15 highly relevant features, which were subsequently used for model development.

6.8.4. Observations

The implementation of Pearson Correlation Coefficient feature selection successfully reduced the dimensionality of the dataset while retaining the most informative network traffic features. The reduction in feature space decreased computational requirements during model training and simplified the learning process for both machine learning and deep learning models.

The selected feature subset served as the common input for the Random Forest, Artificial Neural Network, and Convolutional Neural Network classifiers. Consequently, all three models were trained and evaluated using an identical feature set, ensuring a fair and consistent comparison of their intrusion detection performance.

6.9. RANDOM FOREST

Random Forest (RF) is chosen as the initial classifier for this work to lay the foundation for machine learning-based intrusion detection approach. Random Forest is an ensemble method that integrates the outputs of multiple decision trees to increase the accuracy of classification as well as to decrease the risk of overfitting. With such characteristics as robustness, efficiency of dealing with high-dimensional data, and high learning performance, Random Forest became widely used in the field of intrusion detection.

In the present implementation, the Random Forest classifier was trained using the feature subset obtained after Pearson Correlation Coefficient (PCC) based feature

selection. Prior to model training, the cleaned dataset was divided into training and testing subsets to ensure that the classifier was evaluated using previously unseen data. This approach provides a reliable assessment of the model's ability to generalize to new network traffic instances.

Random forest algorithm trains several decision trees by using random subsets of the training set and features. In the classification stage, each of the trained decision trees separately classifies the network traffic as either malicious or normal. Majority voting is performed to get the final output, and that is the class that gets the highest number of votes from the decision trees.

After training, the model was evaluated on the testing dataset using standard classification metrics, including Accuracy, Precision, Recall, F1-Score, and the Confusion Matrix. These metrics provide a comprehensive assessment of the classifier's ability to correctly identify malicious network traffic while minimizing false classifications.

The implementation of the Random Forest model serves as a benchmark for comparing the performance of traditional machine learning algorithms with deep learning models such as Artificial Neural Networks (ANN) and Convolutional Neural Networks (CNN). The results obtained from this model are presented and discussed in the Results chapter.

Library	Purpose
Scikit-learn	Random Forest implementation
Pandas	Dataset handling
NumPy	Numerical operations
Matplotlib	Result visualization
Joblib	Model serialization

Table 6.3. Libraries used for Random Forest

The implementation generated the following outputs:

- Trained Random Forest model.
- Predicted class labels.
- Confusion Matrix.
- Classification Report.
- Accuracy, Precision, Recall, and F1-Score.
- Model execution statistics.

These outputs were stored for comparative analysis with the ANN and CNN models.

6.10. ARTIFICIAL NEURAL NETWORK (ANN)

Artificial Neural Networks (ANNs) are widely used in intrusion detection systems because of their ability to learn complex and non-linear relationships present in network traffic data. Unlike traditional machine learning algorithms, ANNs automatically learn feature representations through multiple interconnected layers, enabling them to effectively distinguish between normal and malicious network activities.

In the present work, an Artificial Neural Network was implemented as one of the classification models to evaluate the effectiveness of deep learning for IoT intrusion detection. The ANN model was trained using the feature subset obtained after Pearson Correlation Coefficient (PCC)-based feature selection. Using the reduced feature set decreased computational complexity while retaining the most informative attributes required for accurate classification.

Before model training, the processed dataset was divided into training and testing subsets. The ANN architecture consisted of an input layer that received the selected network traffic features, one or more hidden layers responsible for learning complex feature representations, and an output layer that performed binary classification by predicting whether a network traffic instance represented normal or malicious activity.

In the training phase, the model used the backpropagation method and an optimization method to minimize the classification error by adjusting the internal weights. Several training iterations were conducted until convergence was attained and learning

performance stabilized. The performance of the ANN classifier was assessed based on Accuracy, Precision, Recall, F1-Score, and the Confusion Matrix using the test set.

In the training procedure, the model was made to adjust its weight internally through the use of backpropagation and an optimization technique in order to reduce the classification error. The process involved many training epochs until convergence was reached by the model. In order to evaluate the performance of the ANN classifier after training, a test set was used with the following performance measures: Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.

The implementation of the ANN model provides a deep learning baseline for comparison with the Random Forest classifier and the Convolutional Neural Network implemented in this project.

Library	Purpose
TensorFlow	Deep learning framework
Keras	ANN model development
NumPy	Numerical computation
Pandas	Dataset manipulation
Scikit-learn	Data splitting and evaluation
Matplotlib	Training visualization

Table 6.4. Libraries used for ANN

The ANN implementation generated the following outputs:

- Trained ANN model.
- Predicted binary class labels.
- Training and validation loss.
- Training and validation accuracy.
- Confusion Matrix.
- Classification Report.

These outputs were saved for comparison with the Random Forest and Convolutional Neural Network models.

The Artificial Neural Network successfully learned the underlying patterns present in the selected network traffic features and produced binary predictions for normal and malicious network activities. The model demonstrated the capability of deep learning techniques to capture complex relationships within IoT network traffic. The final performance of the ANN model was evaluated using standard classification metrics and later compared with the Random Forest and Convolutional Neural Network models to determine the most effective approach for IoT intrusion detection.

6.11. CONVOLUTIONAL NEURAL NETWORK (CNN)

A Convolutional Neural Network (CNN) was implemented as the primary deep learning model in this project to classify IoT network traffic into normal and malicious categories. CNNs are capable of automatically learning hierarchical feature representations from input data, making them highly effective for complex classification tasks. The implementation of the CNN model in this work follows the methodology presented in the selected research paper, where the network is trained using the features selected through the Pearson Correlation Coefficient (PCC) technique.

The input to the CNN model consisted of the optimized feature set obtained after PCC-based feature selection. Before training, the dataset was divided into training and testing subsets to ensure that the model was evaluated using previously unseen data. The selected features were then provided as input to the convolutional neural network for learning the underlying patterns associated with normal and malicious network traffic.

The CNN architecture consisted of convolutional layers for extracting important feature patterns, activation functions for introducing non-linearity, pooling layers for reducing feature dimensions, and fully connected layers for performing the final binary classification. During training, the network iteratively updated its weights through backpropagation and optimization algorithms to minimize the classification error.

After training, the CNN model was then tested on the test data set. The classifier would make predictions based on each of the network traffic data and would be analyzed using accuracy measures such as Accuracy, Precision, Recall, F1-Score, and Confusion Matrix. The obtained results would later be compared with the results obtained from Random Forest and Artificial Neural Network algorithms.

Library	Purpose
TensorFlow	Deep learning framework
Keras	CNN model implementation
NumPy	Numerical computations
Pandas	Dataset handling
Scikit-learn	Data preprocessing and evaluation
Matplotlib	Visualization of results

Table 6.5. Libraries Used for CNN

The CNN implementation generated the following outputs:

- Trained CNN model.
- Predicted binary class labels.
- Training and validation accuracy.
- Training and validation loss.
- Confusion Matrix.
- Classification Report.
- Accuracy, Precision, Recall, and F1-Score.

These outputs were recorded for comparison with the Random Forest and Artificial Neural Network models in the Results chapter.

The Convolutional Neural Network successfully learned the complex relationships present within the selected network traffic features and produced binary predictions for normal and malicious network traffic. By automatically extracting discriminative feature representations during training, the CNN provided a robust deep learning approach for intrusion detection. The final performance of the CNN model was evaluated using standard classification metrics and compared with the Random Forest and Artificial Neural Network models to determine the most effective classifier for the proposed intrusion detection framework.

7. RESULTS AND DISCUSSION

7.1. CNN

7.1.1. Architecture

Stage	Module	Output Shape	Kernel	Stride	Parameters	% Total	MACs
0	Conv1D	(None, 15, 96)	4	1	480	0.14	-
1	Conv1D	(None, 15, 64)	3	1	18496	5.45	-
2	Conv1D	(None, 15, 32)	2	1	4128	1.22	-
3	Flatten	(None, 480)	-	-	0	0.0	-
4	Dense	(None, 512)	-	-	246272	72.6	-
5	Dense	(None, 128)	-	-	65664	19.36	-
6	Dense	(None, 32)	-	-	4128	1.22	-
7	Dense	(None, 1)	-	-	33	0.01	-

Fig 7.1. CNN Architecture

7.1.2. Hyperparameter Configuration

Hyperparameter	Value
Input Features	15
Optimizer	Adam
Learning Rate	0.001
Epochs	20
Batch Size	64
Loss Function	Binary Crossentropy
Conv1 Filters	96
Conv1 Kernel	4
Conv2 Filters	64
Conv2 Kernel	3
Conv3 Filters	32
Conv3 Kernel	2
Dense Layer 1	512
Dense Layer 2	128
Dense Layer 3	32
Output Activation	Sigmoid

Fig 7.2. Hyperparameters

7.1.3. Training Performance

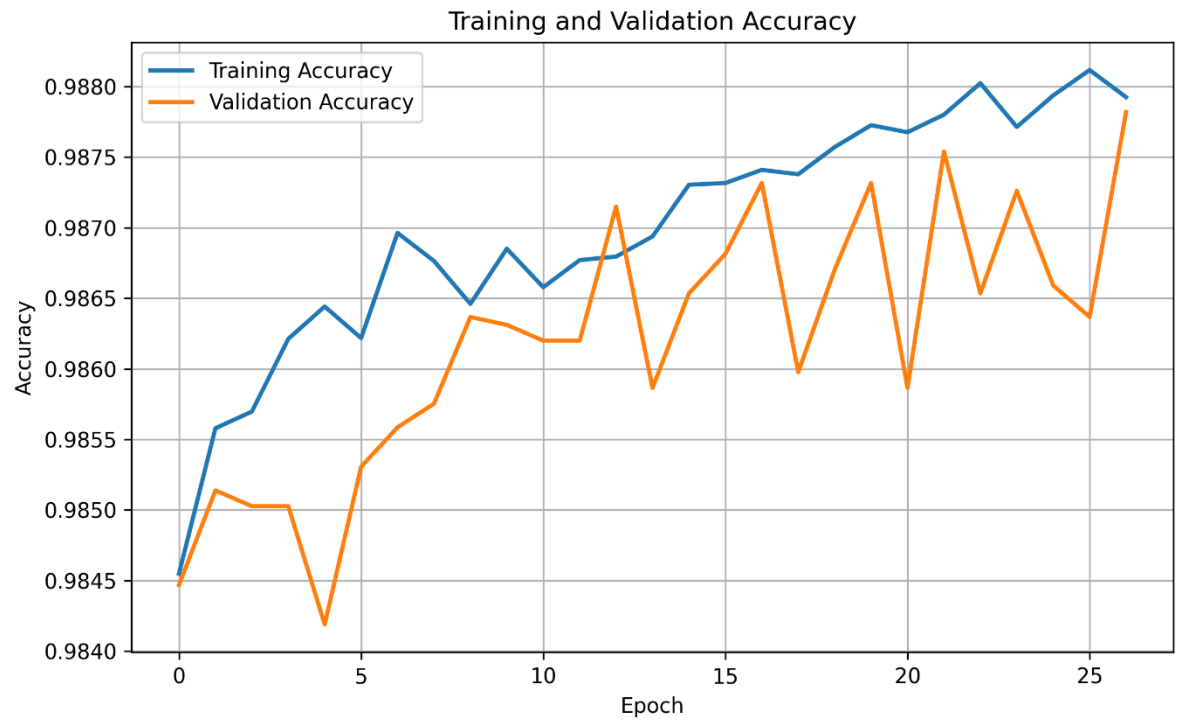


Figure 7.3. Training and Validation Accuracy over epochs

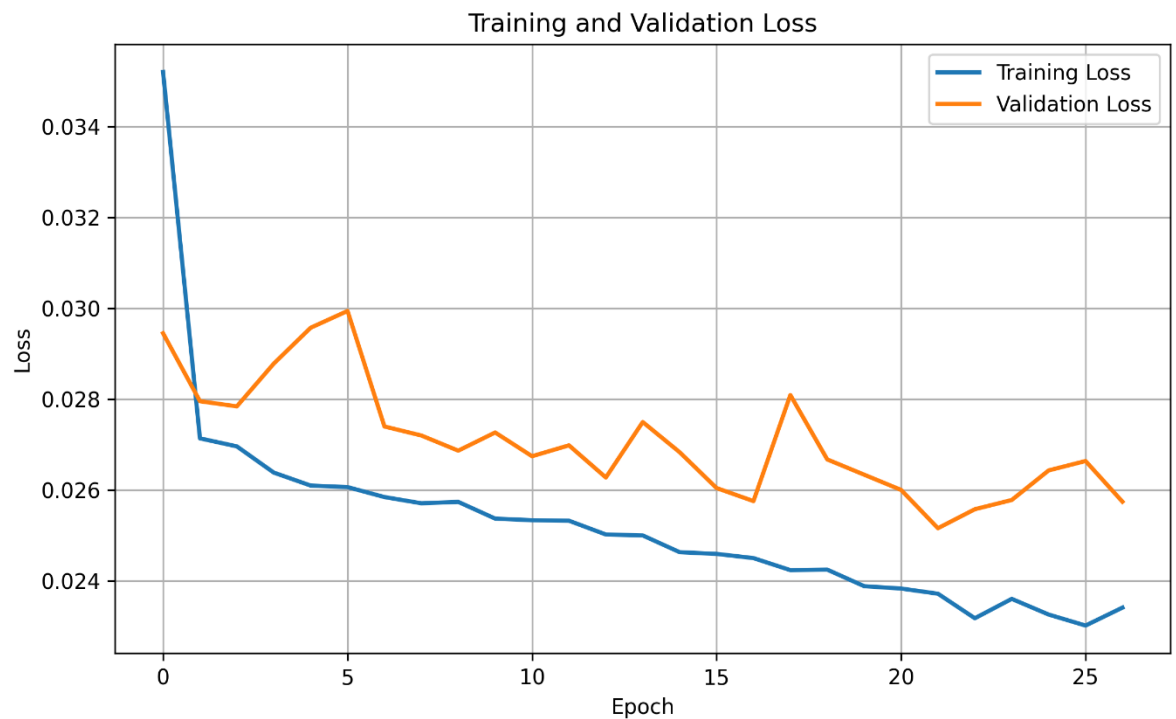


Figure 7.4. Training and Validation Loss over epochs

7.1.4. Cross Validation Results

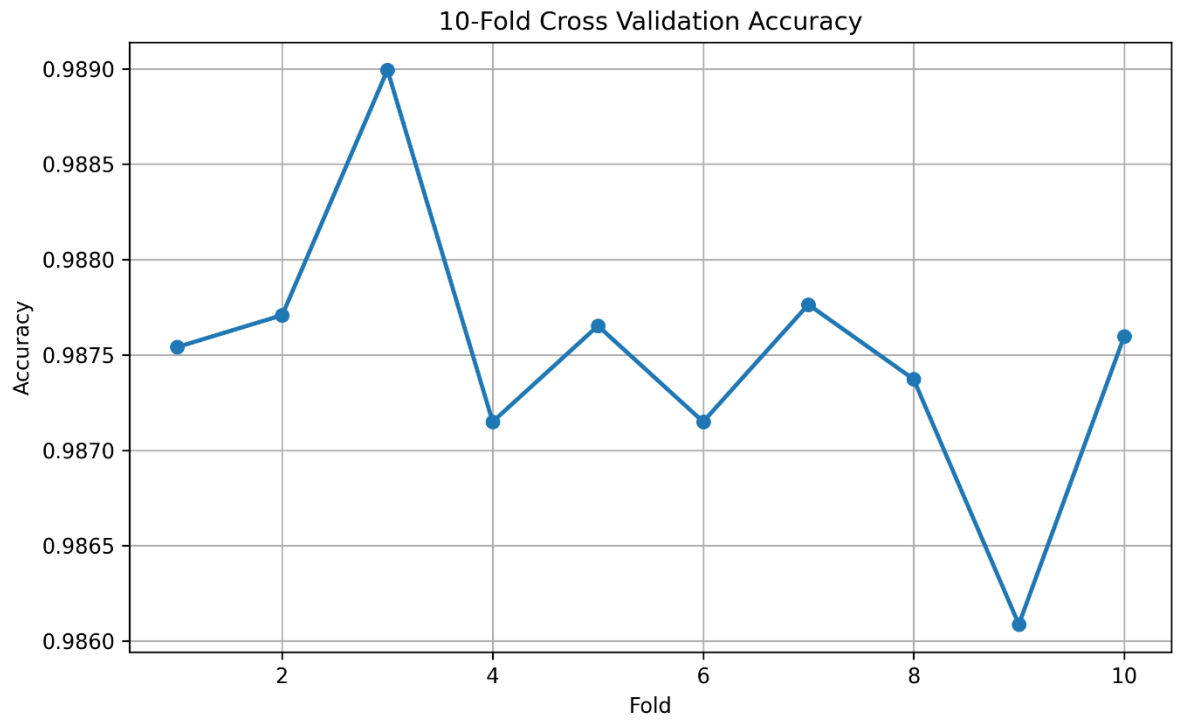


Figure 7.5. Accuracy obtained across 10 Folds

7.1.5. Quantitative Performance

Metric	Mean	Std
Accuracy	0.987501179	0.000717648
Balanced Accuracy	0.876193227	0.012983422
Precision	0.994214858	0.000632967
Sensitivity	0.992979702	0.001015329
F1	0.993596372	0.00037025
Macro F1	0.86693722	0.006469721
AUC	0.995647856	0.000363316
Specificity	0.759406751	0.026656187

Table 7.1. Evaluation Metrics

7.1.6. ROC Analysis

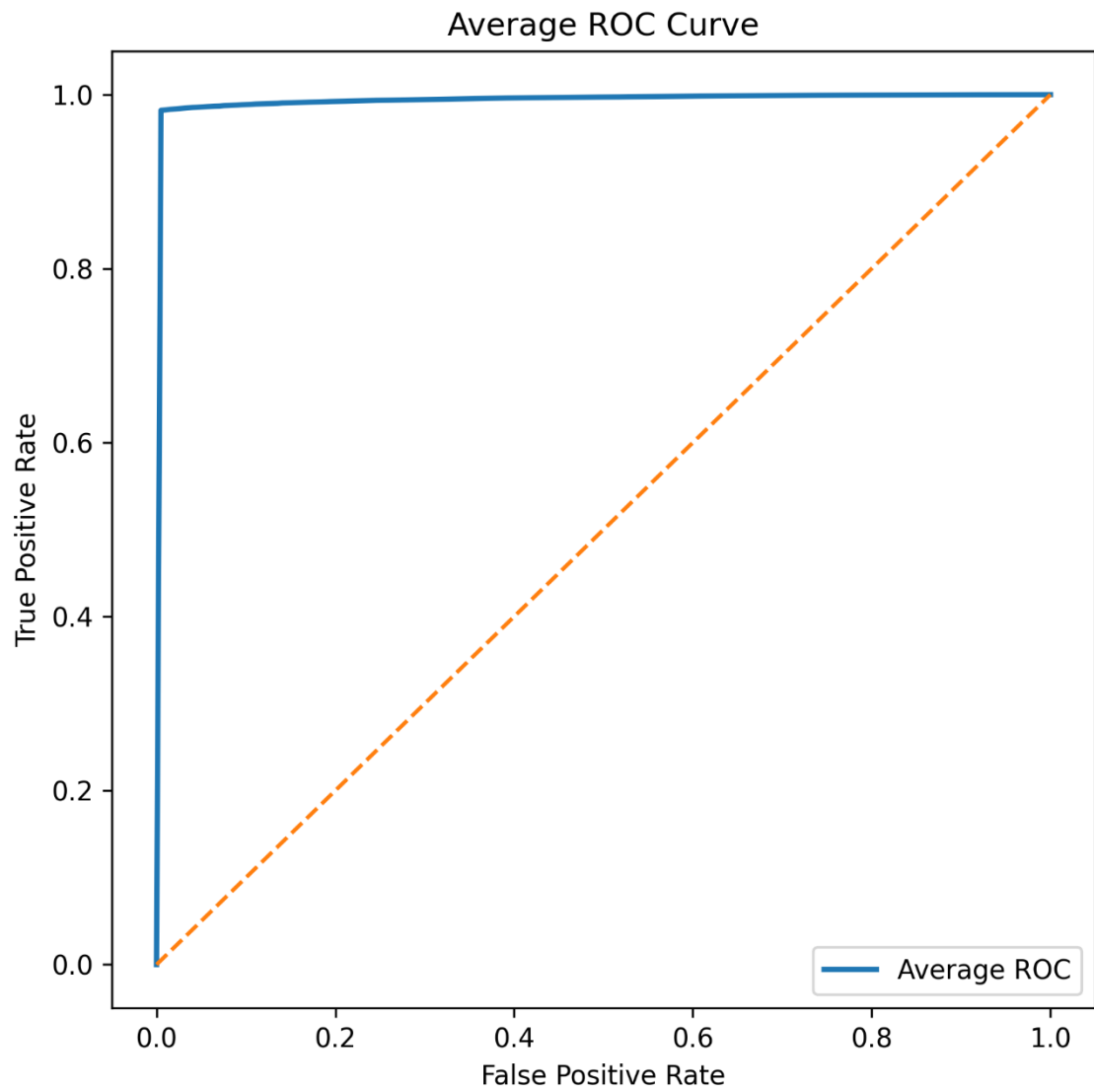


Figure 7.6. Average ROC curve across the 10 – fold curve.

7.1.7. Confusion Matrix

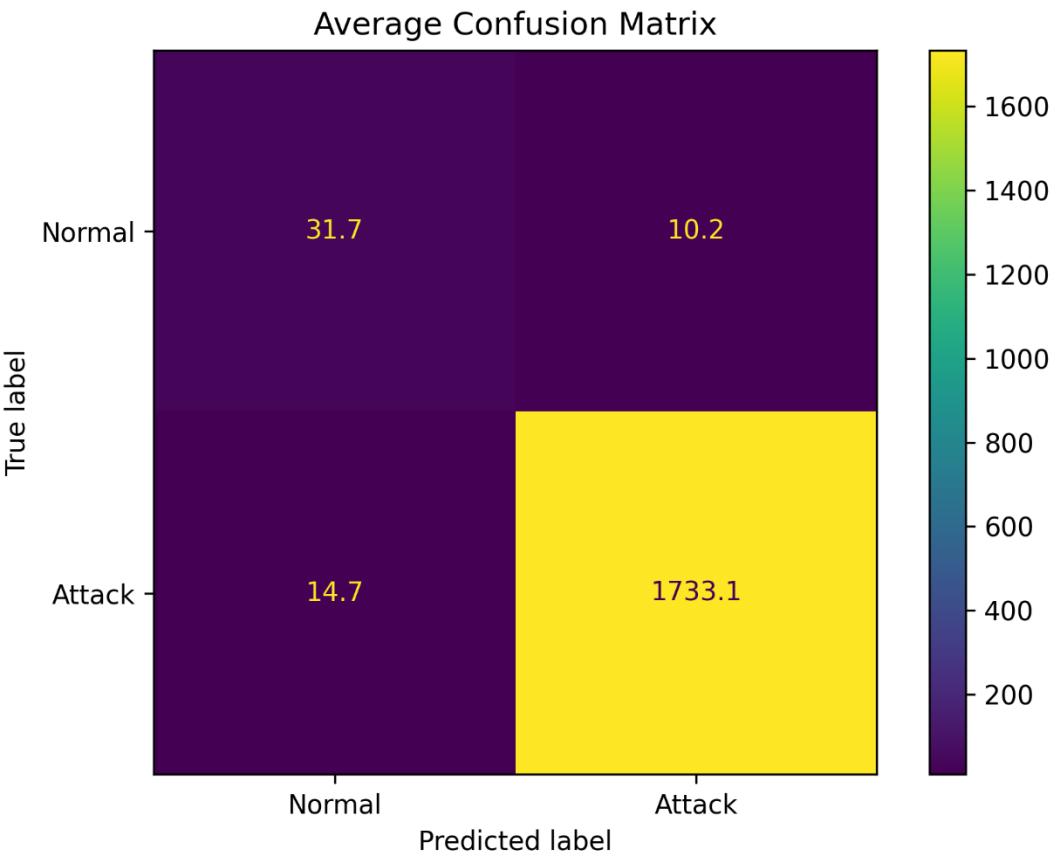


Figure 7.7. Average Confusion Matrix over 10 validation folds.

7.1.8. Average Metric Comparison

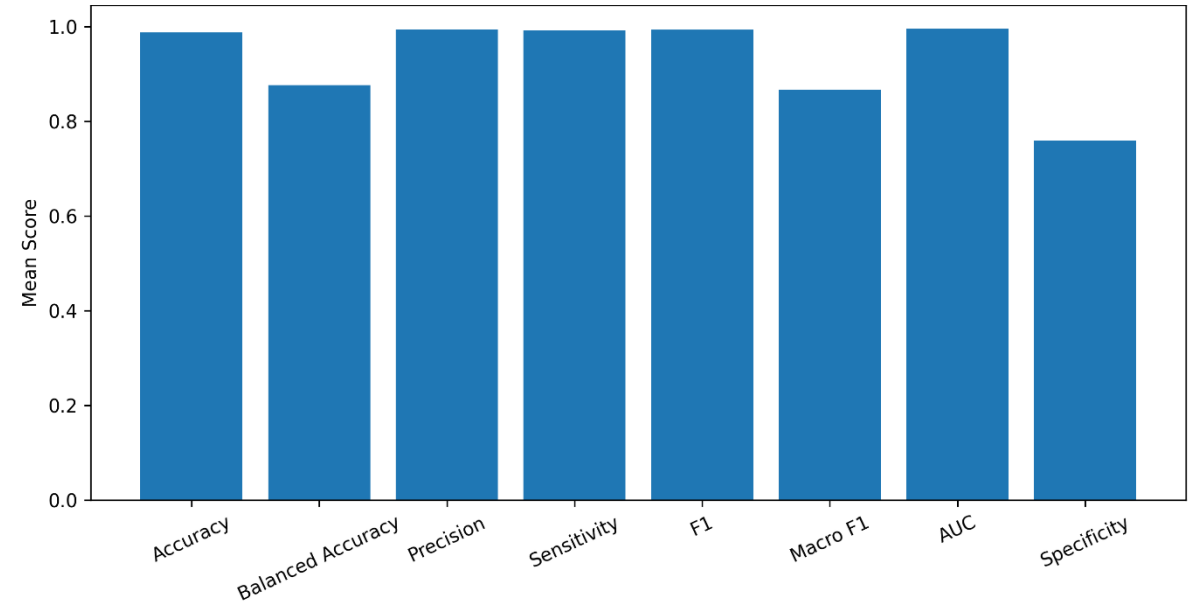


Figure 7.8. Metric Comparison

7.1.9. t - SNE

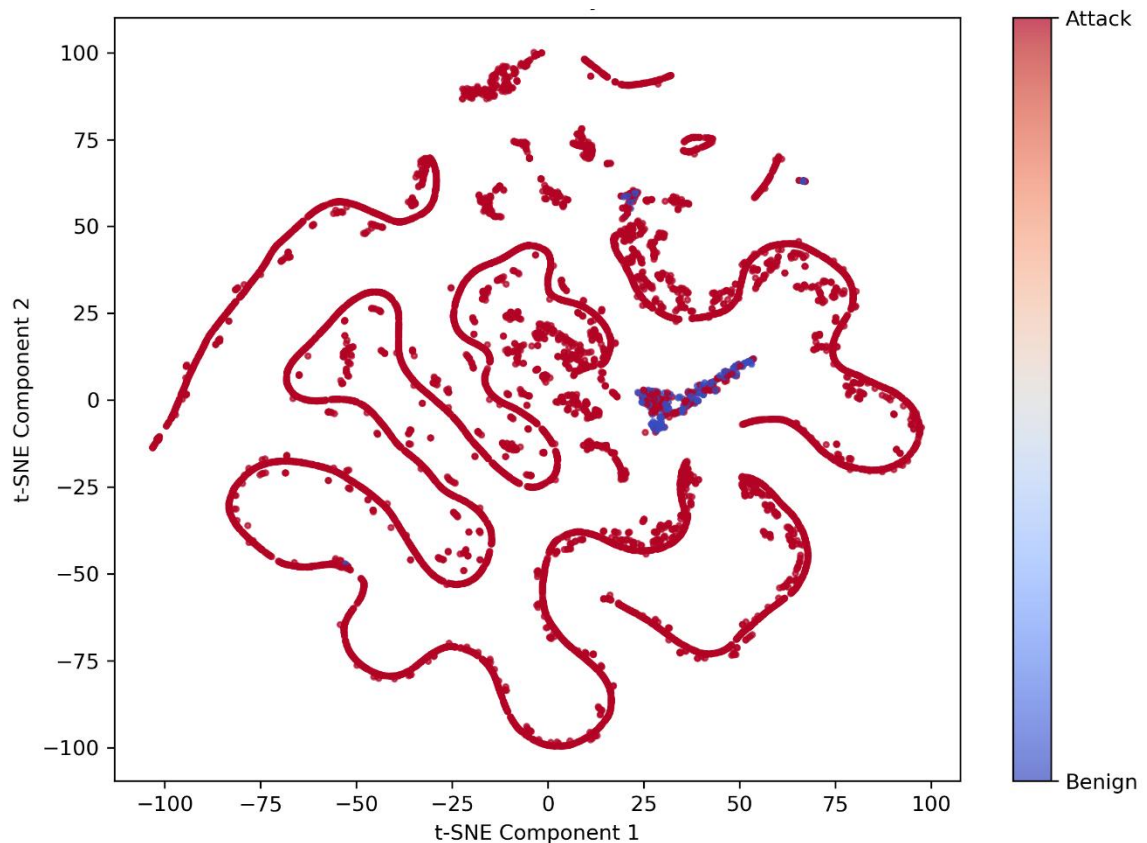


Figure 7.9 Two-dimensional t-SNE visualization of the binary intrusion dataset

7.1.10. Discussion

In terms of binary intrusion detection, the Convolutional Neural Network (CNN) gave an outstanding performance of 98.75% for average accuracy, 99.42% for precision, 99.30% for sensitivity (or recall), 99.36% for F1-score, and 99.56% for the area under curve (AUC) in 10-fold cross-validation. The low standard deviation for all metrics means that the model generated stable and reliable predictions in all folds. The ROC curve stays close to the upper left corner, which proves good discrimination power between benign and malicious traffic. The average confusion matrix showed that the CNN correctly classified most attack examples with minimal false positives and false negatives. The lower specificity (75.94%) as compared to sensitivity shows that the classification of benign traffic was more difficult because of the class imbalance issue in the data, when attacks were much more than normal. The convolutional layers successfully generated local feature patterns of the selected network features, while the dense layers made high-level classification.

7.2. ANN

7.2.1. Architecture

Stage	Module	Output Shape	Activation	Parameters
0	Input	(15)	-	0
1	Dense	(128)	ReLU	2048
2	Dense	(64)	ReLU	8256
3	Dense	(32)	ReLU	2080
4	Dense	(1)	Sigmoid	33

Figure 7.10 ANN Architecture

7.2.2. Hyperparameter Configuration

Hyperparameter	Value
Input Features	15
Optimizer	Adam
Learning Rate	0.001
Epochs	20
Batch Size	64
Loss Function	Binary Crossentropy
Dense Layer 1	128 ReLU
Dense Layer 2	64 ReLU
Dense Layer 3	32 ReLU
Output Layer	Sigmoid

Figure 7.11. Hyperparameters

7.2.3. Training Accuracy

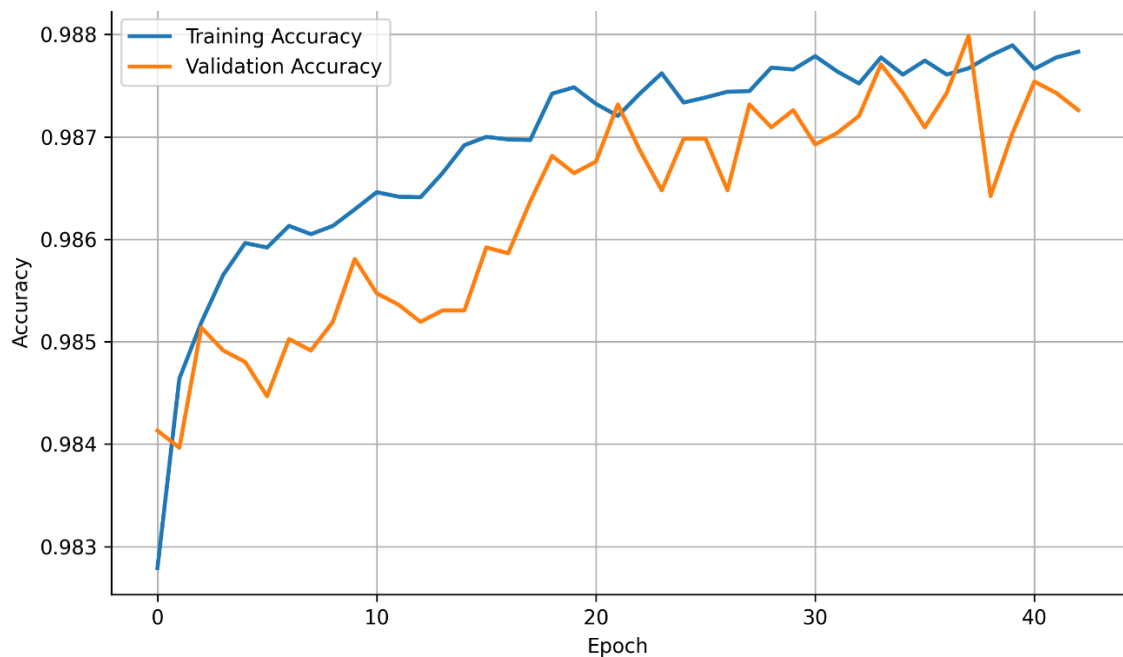


Figure 7.12 Average Training and Validation across 10-fold cross validation

7.2.4. Training Loss

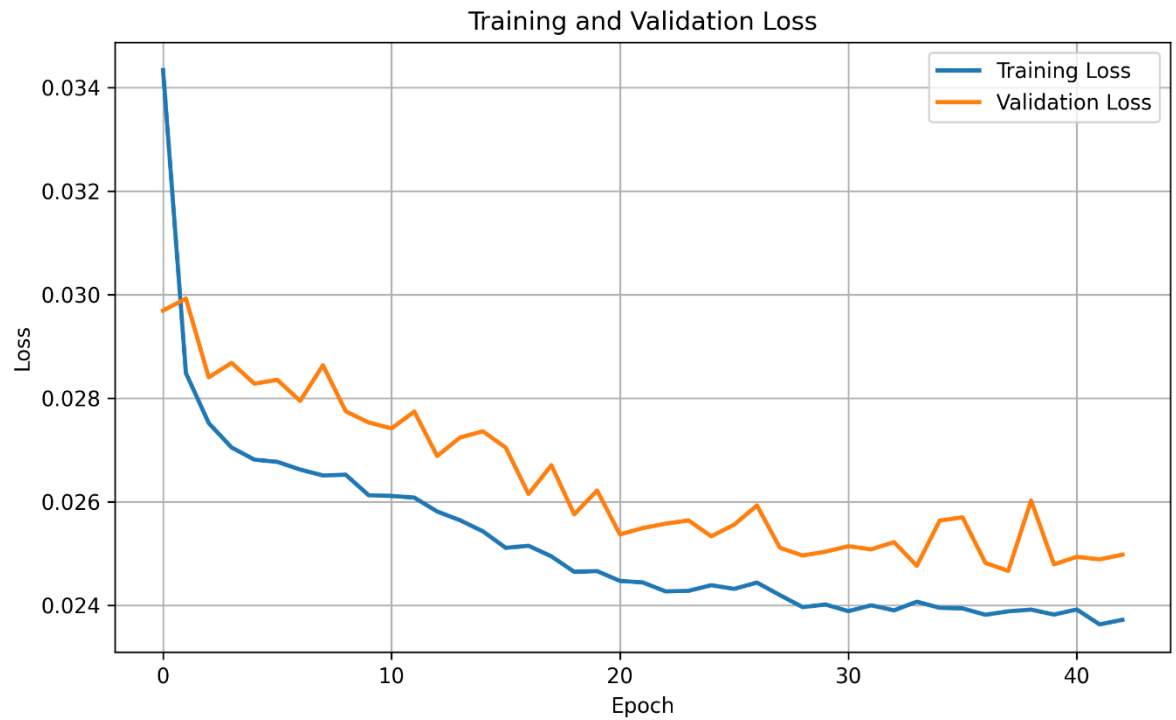


Figure 7.13 Average Training and Validation Loss over 10-fold cross validation

7.2.5. Fold Accuracy

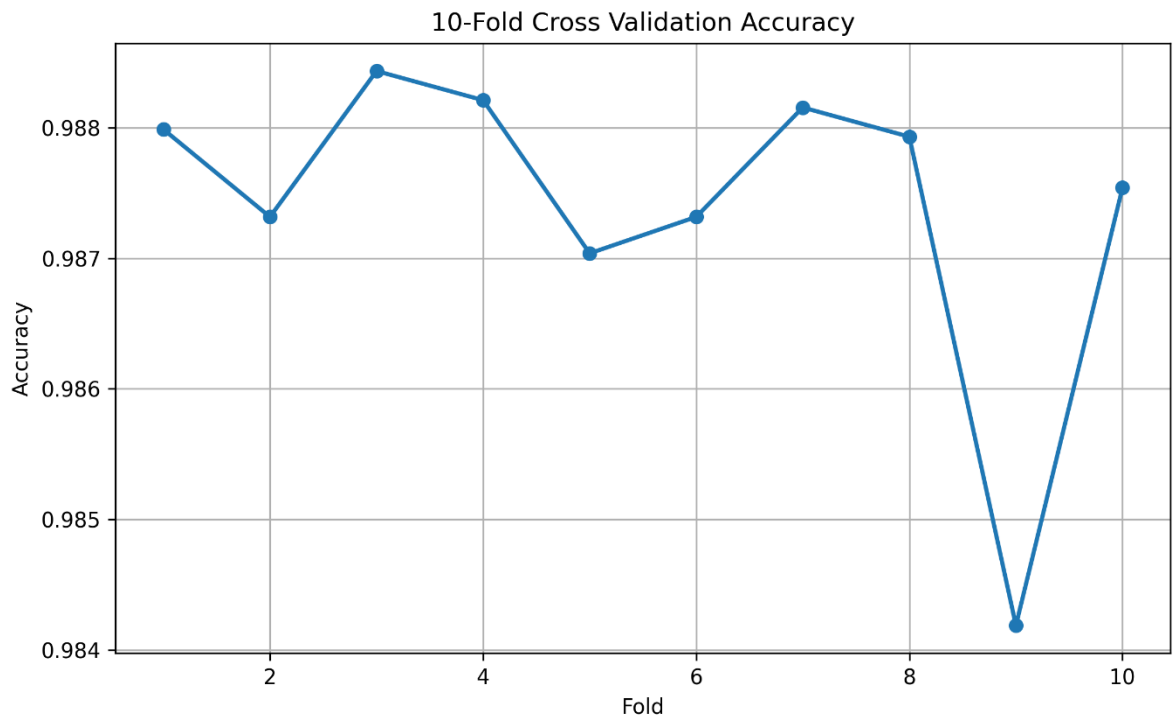


Figure 7.14. Fold Accuracy over 10 Folds

7.2.6. Performance Metrics

Experiment Summary

Metric	Mean	Std
Accuracy	0.987489995723221	0.0009859214401362
Balanced Accuracy	0.889203273944948	0.0198802791359088
Precision	0.994850366280784	0.0009605572347773
Recall	0.99232744799218	0.0010957775755908
F1	0.9935865752207136	0.0005058954377463
Macro F1	0.8700348719566142	0.010898940709801
AUC	0.995517999572184	0.0005852016192289
Specificity	0.7860790998977156	0.0403228720402634

Figure 7.15. Evaluation Metrics

7.2.7. ROC Curve

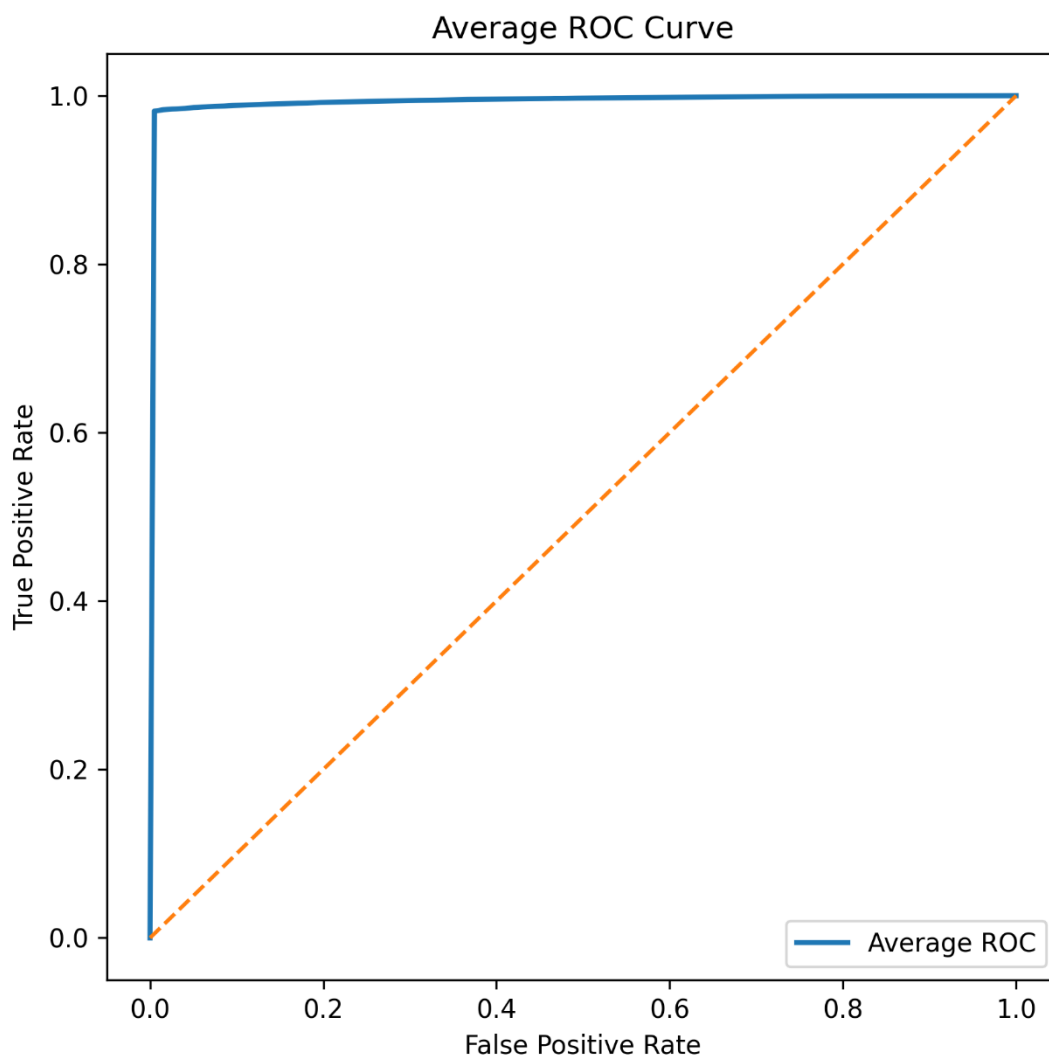


Figure 7.16 Average ROC curve

7.2.8. Confusion Matrix

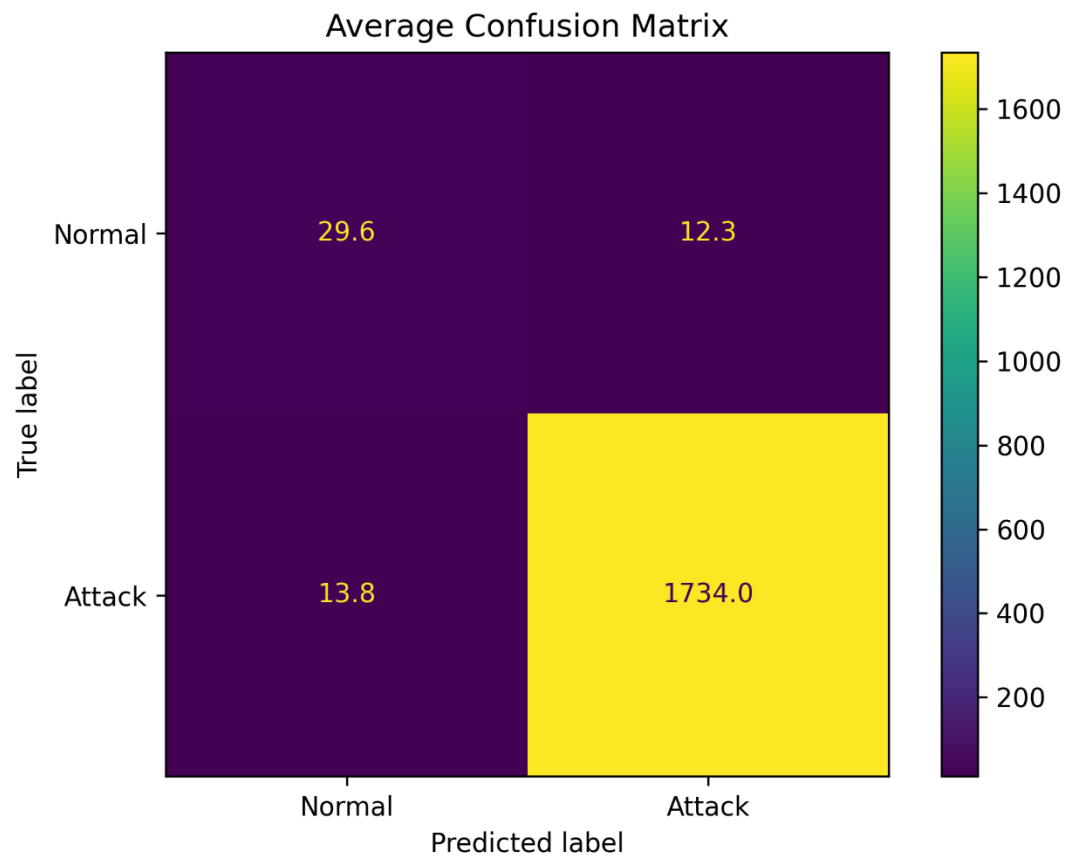


Figure 7.17 Average Confusion Matrix over 10 Folds

7.2.9. Metric Comparison

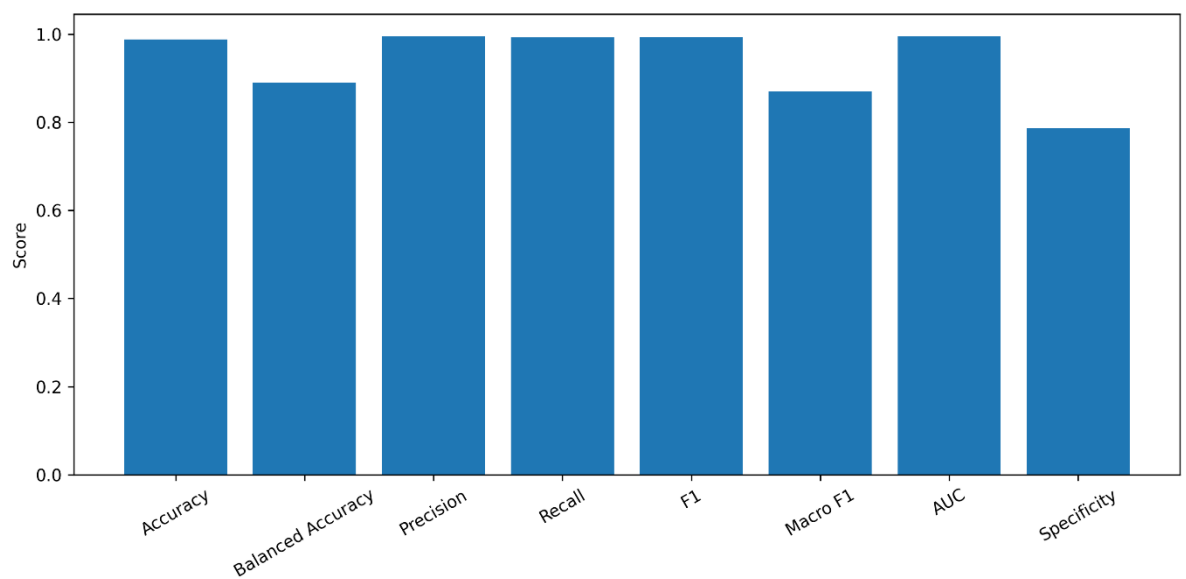


Figure 7.18 Metric Comparison

7.2.10. Discussion

The Artificial Neural Network (ANN) produced an average performance of 98.75% accuracy, 99.49% precision, 99.23% sensitivity, 99.36% F1-Score, and 99.55% AUC, thus exhibiting high-performance characteristics in binary classification. The low standard deviation seen in the performance measures shows that the learning is consistent during cross validation. The ROC plot nearly touches the perfect classifier, showing that the network has effectively classified attack traffic from benign traffic at different levels of threshold. The confusion matrix shows that most attack instances have been identified, and only a few benign instances have been misclassified. Although the ANN has achieved higher specificity (78.61%) compared to the CNN, the accuracy and F1-Score of the two algorithms is similar. The fully connected network was capable of learning nonlinear interactions between the selected PCC features.

7.3. Random Forest

7.3.1. Hyperparameter

Hyperparameter	Value
Algorithm	Random Forest Classifier
Number of Trees	100
Random State	42
CPU Cores	-1 (All Available Cores)
Classification	Binary
Decision Threshold	0.1
Cross Validation	Stratified 10-Fold
Number of Folds	10
Feature Selection	PCC (15 Features)
Feature Scaling	StandardScaler

Table 7.2. Hyperparameters for Random Forest

7.3.2. Fold Accuracy

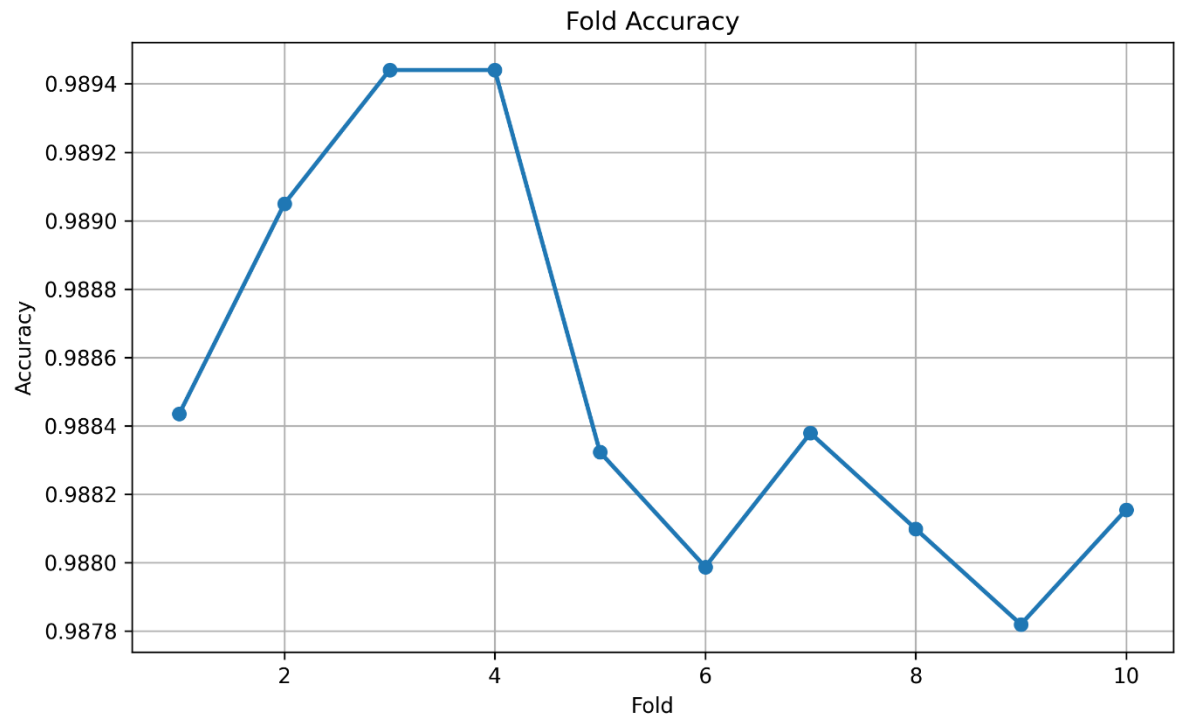


Figure 7.19 Accuracy over 10 Folds

7.3.3. Experiment Summary

Metric	Mean	Std
Accuracy	0.988512482	0.000589248
Balanced Accuracy	0.879030782	0.012734442
Precision	0.994333787	0.000612124
Recall	0.993900865	0.000666847
F1	0.99411699	0.00030196
Macro F1	0.875666644	0.006801352
AUC	0.995457652	0.001926717
Specificity	0.7641607	0.025835273

Table 7.3. Evaluation Metrics

7.3.4. Average ROC

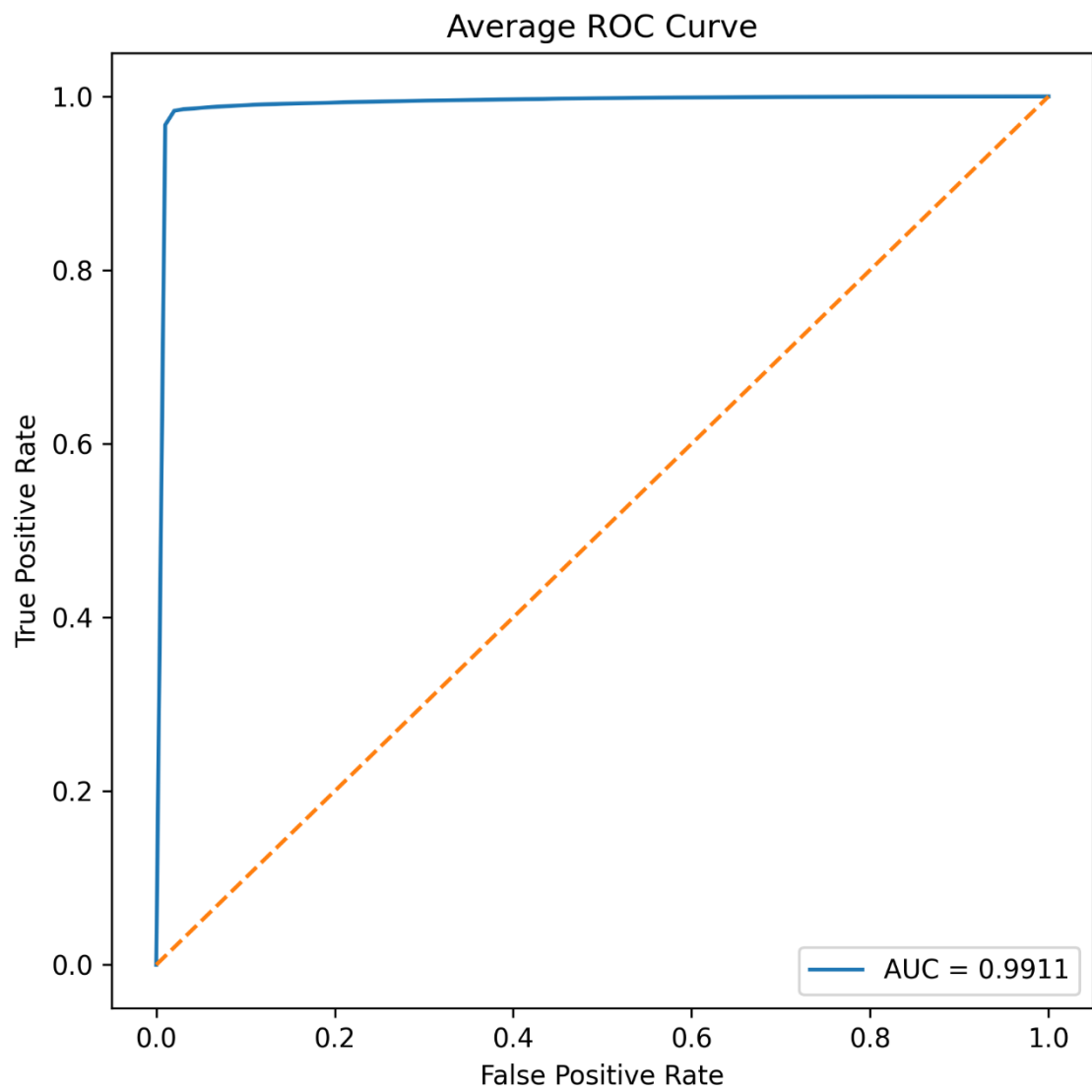


Figure 7.20 Average ROC curve of Random Forest

7.3.5. Average Confusion Matrix

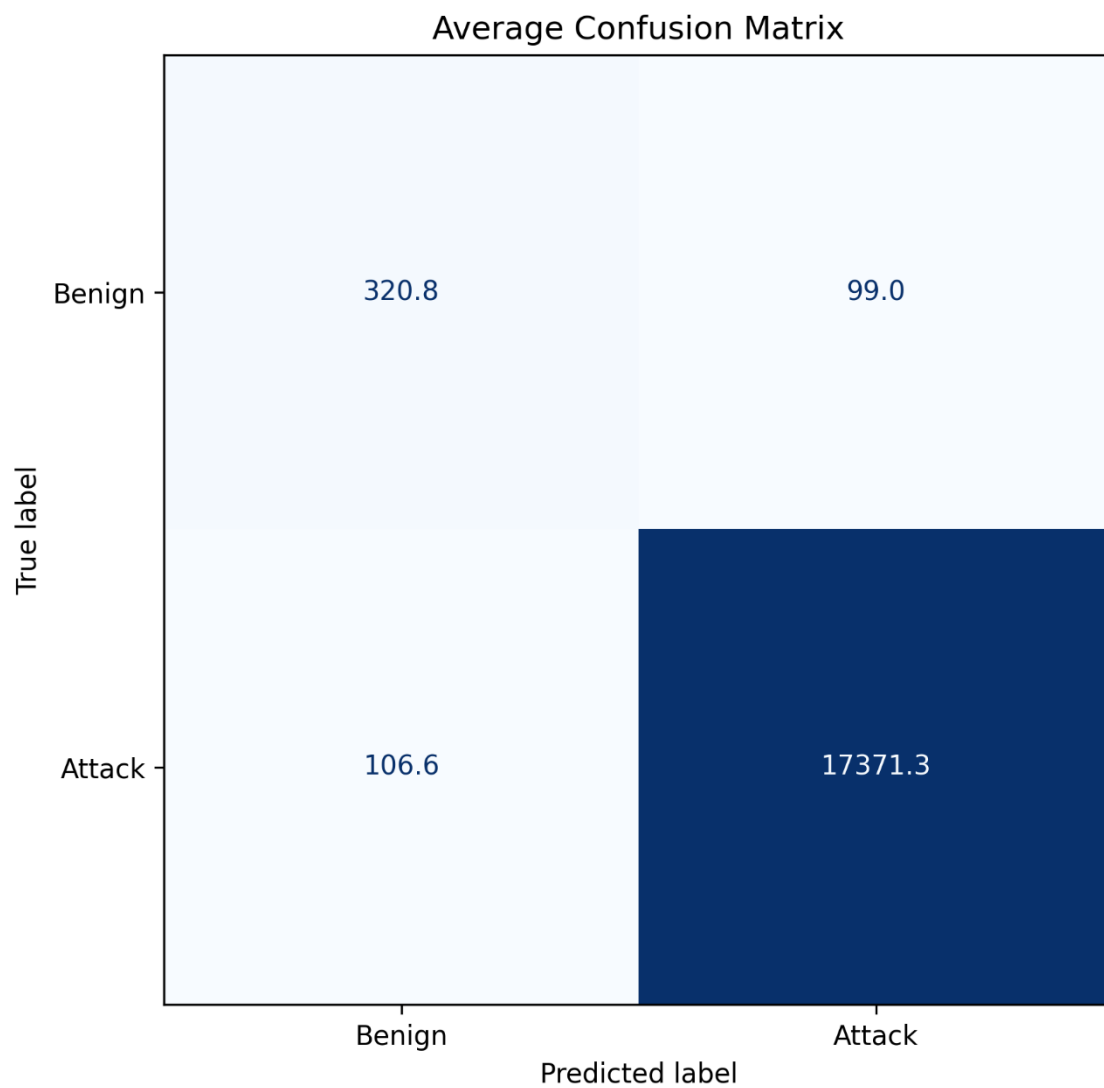


Figure 7.21 Average Confusion Matrix for Random Forest

7.3.6. Metric Comparison

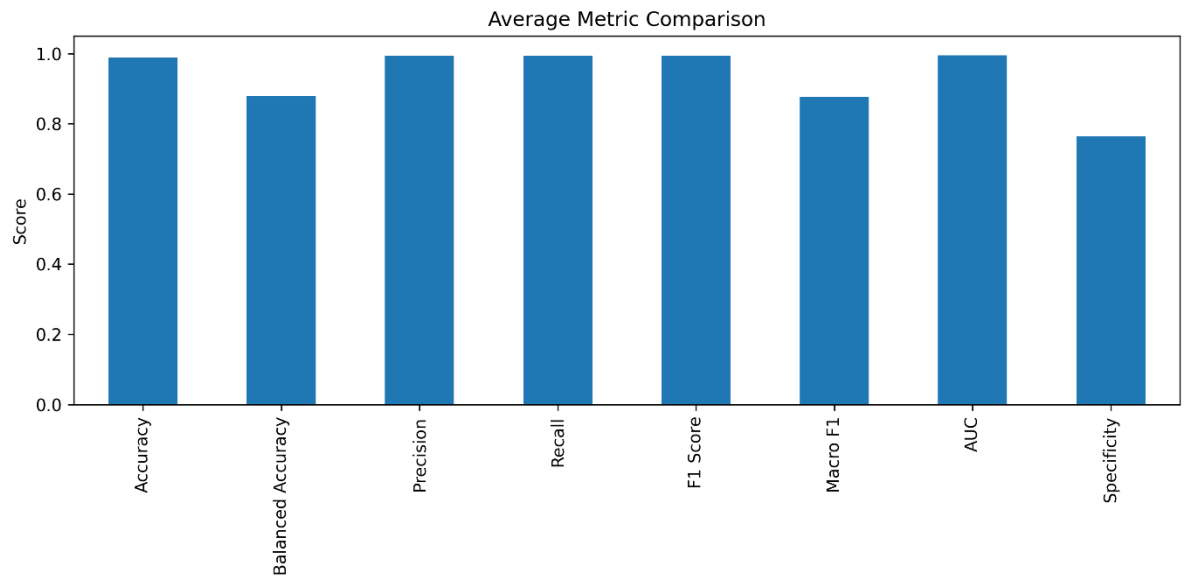


Figure 7.22. Average Metric Comparison

7.3.7. Discussion

Random Forest Classifier showed the best overall results among all the tested models, with an average accuracy of 98.85%, precision of 99.43%, recall of 99.39%, F1-score of 99.41% and an AUC of 99.55%. Stable values of the standard deviation for all metrics confirm the effectiveness and reliability of the proposed ensemble learning method. The ROC curve still keeps its distance from the ideal classifier, showing good separability of benign and malicious traffic. From the confusion matrix, it follows that most of attacks were recognized properly, with almost no false negatives and false positives. In comparison with the deep learning models, specificity (76.42%) is still not that high due to an imbalance of data set; nevertheless, the voting technique used in ensembles reduced classification mistakes.

7.4. COMPARISION

Metric	CNN	ANN	RF
Accuracy	98.75	98.75	98.85
Balanced Accuracy	87.62	88.92	87.90
Precision	99.42	99.49	99.43
Recall/Sensitivity	99.30	99.23	99.39
F1	99.36	99.36	99.41
Macro F1	86.69	87.00	87.57
AUC	99.56	99.55	99.55
Specificity	75.94	78.61	76.42

Table 7.4. Model Comparison

7.4.1. Comparative Discussion

Overall, the performance of all three models was exceptionally good, achieving accuracies over 98.7% and AUC scores over 99.5%, suggesting an excellent performance in terms of binary intrusion detection. The Random Forest classifier showed the best overall accuracy and F1-scores, whereas the ANN yielded the best specificity and proved to be slightly better in distinguishing the benign traffic. The CNN model provided performance similar to the other two while learning discriminative features by itself using the convolutions. Although specificity was still less than the sensitivity for all models because of the extremely imbalanced classes in the CICIoT2023 dataset, all three models managed to provide a high detection rate of attacks with a minimum of false negative detections.

8. CONCLUSION

In this project, the implementation of an anomaly-based Intrusion Detection System (IDS) for IoT networks through machine learning and deep learning algorithms is carried out. The increased use of IoT technology poses many challenges to cybersecurity because of the high number of traffic in a heterogeneous environment and the increased complexity of cyberattacks. In order to counter this challenge, the design of the IDS through the methodology in the chosen research paper is done.

The implementation began with the collection and preprocessing of the CICIoT2023 dataset. Since the original dataset consisted of a large number of network traffic records distributed across multiple files, the data was merged, sampled, cleaned, and transformed into a binary classification dataset. Exploratory Data Analysis (EDA) was then performed to understand the characteristics of the dataset and validate its suitability for machine learning and deep learning applications.

To reduce data dimensionality and eliminate redundant attributes, Pearson Correlation Coefficient (PCC)-based feature selection was applied. The selected feature subset was subsequently used to train three classification models, namely Random Forest (RF), Artificial Neural Network (ANN), and Convolutional Neural Network (CNN). Implementing multiple classifiers enables a comparative evaluation of traditional machine learning and deep learning approaches for IoT intrusion detection.

The implementation presented in this work establishes a complete intrusion detection framework, beginning with raw network traffic data and progressing through preprocessing, feature selection, model development, and evaluation. The experimental analysis and comparative performance of the implemented models are presented in the Results chapter, providing a detailed assessment of their effectiveness in detecting malicious network traffic.

Overall, the work demonstrates a structured implementation of an anomaly-based intrusion detection framework for IoT environments and provides a foundation for further improvements in intelligent network security systems.

9. SCOPE FOR FUTURE WORK

Despite the fact that the developed intrusion detection system offers a methodological way of detecting malicious network traffic in the Internet of Things setting, there are still multiple possibilities for its future development.

Currently, the classification in the system is based on the two-class model that includes classifying the network traffic as being normal or malicious. Future developments of the system could lead to multi-class classification and the possibility of identifying particular classes of attacks including DDoS, spoofing, reconnaissance, brute force attacks, among others.

The current study utilizes Pearson Correlation Coefficient (PCC) for feature selection. Future research may investigate alternative feature selection techniques such as Mutual Information, Recursive Feature Elimination (RFE), Principal Component Analysis (PCA), or hybrid feature selection methods to evaluate their impact on detection performance and computational efficiency.

Additional machine learning and deep learning algorithms, including Long Short-Term Memory (LSTM), Gated Recurrent Units (GRU), Transformer-based architectures, and hybrid ensemble models, may also be explored for improving intrusion detection accuracy and robustness.

The framework can further be evaluated using additional benchmark intrusion detection datasets, such as BOT-IoT, UNSW-NB15, TON-IoT, CICIDS2017, NSL-KDD, and IoTID20, to assess its generalization capability across diverse IoT environments.

Finally, the proposed intrusion detection framework can be integrated into real-time IoT systems and edge computing platforms, enabling continuous monitoring of network traffic and rapid detection of emerging cyber threats with reduced computational latency.

10. REFERENCES

- [1] D. Akgun, S. Hizal, and U. Cavusoglu, "A new DDoS attacks intrusion detection model based on deep learning for cybersecurity," *Computers & Security*, vol. 118, p. 102748, Jul. 2022, doi: 10.1016/j.cose.2022.102748.
- [2] A. D. Vibhute, C. H. Patil, A. V. Mane, and K. V. Kale, "Towards Detection of Network Anomalies using Machine Learning Algorithms on the NSL-KDD Benchmark Datasets," *Procedia Computer Science*, vol. 233, pp. 960–969, 2024, doi: 10.1016/j.procs.2024.03.285.
- [3] S. Fraihat, S. Makhadmeh, M. Awad, M. A. Al-Betar, and A. Al-Redhaei, "Intrusion detection system for large-scale IoT NetFlow networks using machine learning with modified Arithmetic Optimization Algorithm," *Internet of Things*, vol. 22, p. 100819, Jul. 2023, doi: 10.1016/j.iot.2023.100819.
- [4] M. Vishwakarma and N. Kesswani, "A new two-phase intrusion detection system with Naïve Bayes machine learning for data classification and elliptic envelop method for anomaly detection," *Decision Analytics Journal*, vol. 7, p. 100233, Jun. 2023, doi: 10.1016/j.dajour.2023.100233.
- [5] I. Kerrakchou, A. Abou El Hassan, S. Chadli, M. Emharraf, and M. Saber, "Selection of efficient machine learning algorithm on Bot-IoT dataset for intrusion detection in internet of things networks," *Indonesian Journal of Electrical Engineering and Computer Science*, vol. 31, no. 3, pp. 1784–1793, Sep. 2023, doi: 10.11591/ijeecs.v31.i3.pp1784-1793.
- [6] A. Aldaej, I. Ullah, T. A. Ahanger, and M. Atiquzzaman, "Ensemble technique of intrusion detection for IoT-edge platform," *Scientific Reports*, vol. 14, art. no. 12066, May 2024, doi: 10.1038/s41598-024-62435-w.
- [7] M. Subramanian, K. Shanmugavadivel, P. S. Nandhini, and R. Sowmya, "DDoS Attack Detection Using Deep Learning and Bayesian Hyperparameter Optimization," in *Proceedings of 7th International Conference on Harmony Search, Soft Computing and Applications, Lecture Notes on Data Engineering and Communications Technologies*, vol. 140, Springer, 2022, pp. 395–404, doi: 10.1007/978-981-19-2948-9_38.

- [8] X. Gao, C. Shan, C. Hu, Z. Niu, and Z. Liu, "An Adaptive Ensemble Machine Learning Model for Intrusion Detection," *IEEE Access*, vol. 7, pp. 82512–82521, 2019, doi: 10.1109/ACCESS.2019.2923640.
- [9] X. Gao, C. Shan, C. Hu, and Z. Liu, "Anomaly Network Monitoring for Software-Defined Networking," in *Proceedings of the 5th International Conference on Electronic Information Engineering and Computer Communication (EIECC)*, 2025, pp. 1–6.
- [10] N. G. Pardeshi and D. V. Patil, "Binary and Multiclass Classification Intrusion Detection System using Benchmark NSL-KDD and Machine Learning Models," in *Proceedings of the International Conference on Data Science and Network Security (ICDSNS)*, 2024, pp. 1–6.
- [11] Amali, A. A. Sulaeman, A. M. Rifa'i, S. Raharjo, and E. Utami, "Detecting DDoS Attacks Through Decision Tree Analysis: An EDA Approach with the CIC DDoS 2019 Dataset," in *Proceedings of the International Conference on Information Technology and Computer Science*, 2024, pp. 1–6.
- [12] A. Alsarhan, M. Barhoush, B. Khassawneh, M. Al-Essa, M. Aljaidi, and Q. Al-Na'amneh, "Deep Learning Utilization for DDoS Attack Detection with Federated Learning: A Case Study on the CICDDoS2019 Dataset," *Engineering, Technology & Applied Science Research*, vol. 16, no. 1, pp. 31203–31208, Feb. 2026, doi: 10.48084/etasr.14119.
- [13] A. Hinojosa and N. E. Majd, "Edge Computing Network Intrusion Detection System in IoT Using Deep Learning," in *Proceedings of the IEEE International Conference on Computer Science and Information Systems*, 2024, pp. 1–6.
- [14] A. Berguiga, A. Harchay, and A. Massaoudi, "HIDS-IoMT: A Deep Learning-Based Intelligent Intrusion Detection System for the Internet of Medical Things," *IEEE Access*, vol. 13, pp. 32145–32158, 2025, doi: 10.1109/ACCESS.2025.3543127.
- [15] N. Hassouneh and S. al-sharaeh, "Intrusion Detection in IoT Networks using LSTM Deep Learning Models with the UNSW-NB15 Dataset," in *Proceedings of the IEEE International Conference on Platform Technology and Service*, 2024, pp. 1–6.

- [16] H. Mliki, A. H. Kaceam, and L. Chaari, "Machine Learning for Attack Detection in IoT Networks," *EAI Endorsed Transactions on Security and Safety*, vol. 8, no. 29, pp. 1–12, 2023.
- [17] J. Du, K. Yang, Y. Hu, and L. Jiang, "NIDS-CNNLSTM: Network Intrusion Detection Classification Model Based on Deep Learning," *IEEE Access*, vol. 11, pp. 24808–24821, 2023, doi: 10.1109/ACCESS.2023.3254915.
- [18] T.-T.-H. Le, R. W. Wardhani, D. S. C. Putranto, U. Jo, and H. Kim, "Toward Enhanced Attack Detection and Explanation in Intrusion Detection System-Based IoT Environment Data," *IEEE Access*, vol. 11, pp. 131661–131676, 2023, doi: 10.1109/ACCESS.2023.3336678.
- [19] U. C. Akuthota and L. Bhargava, "Transformer-Based Intrusion Detection for IoT Networks," *IEEE Internet of Things Journal*, vol. 12, no. 4, pp. 4432–4442, Feb. 2025, doi: 10.1109/JIOT.2025.3525494.
- [20] J. Ashraf, G. M. Raza, B.-S. Kim, A. Wahid, and H.-Y. Kim, "Making a Real-Time IoT Network Intrusion-Detection System (INIDS) Using a Realistic BoT–IoT Dataset with Multiple Machine-Learning Classifiers," *Applied Sciences*, vol. 15, no. 4, art. no. 2043, Feb. 2025, doi: 10.3390/app15042043.
- [21] M. Iwanowski, D. Olszewski, W. Graniszewski, J. Krupski, and F. Pelc, "The Choice of Training Data and the Generalizability of Machine Learning Models for Network Intrusion Detection Systems," *Applied Sciences*, vol. 15, no. 15, art. no. 8466, Aug. 2025, doi: 10.3390/app15158466.
- [22] S. Yaras and M. Dener, "IoT-Based Intrusion Detection System Using New Hybrid Deep Learning Algorithm," *Electronics*, vol. 13, no. 6, art. no. 1053, Mar. 2024, doi: 10.3390/electronics13061053.
- [23] S.-M. Tseng, Y.-Q. Wang, and Y.-C. Wang, "Multi-Class Intrusion Detection Based on Transformer for IoT Networks Using CIC-IoT-2023 Dataset," *Future Internet*, vol. 16, no. 8, art. no. 284, Aug. 2024, doi: 10.3390/fi16080284.
- [24] S. Dhote and D. Agrawal, "A Comparative Machine Learning Framework for Network Intrusion Detection Using the NSL-KDD Dataset," *International Journal of*

IoT, Embedded Systems and Industrial Automation (IJIESIA), vol. 1, no. 2, art. no. e002, Jul.–Sep. 2026, doi: 10.66261/017smj60.

[25] D. Z. Rodríguez, O. D. Okey, S. S. Maidin, E. U. Udo, and J. H. Kleinschmidt, "Attentive transformer deep learning algorithm for intrusion detection on IoT systems using automatic Xplainable feature selection," PLoS ONE, vol. 18, no. 10, p. e0286652, Oct. 2023, doi: 10.1371/journal.pone.0286652.

[26] S. W. Nourillean, W. Mefteh, and A. M. Frihida, "Intrusion detection system-based ensemble machine learning to improve IoT network against cyber attacks," The Journal of Supercomputing, vol. 82, art. no. 370, 2026, doi: 10.1007/s11227-026-08529-y.

[27] A. Shebl, E. I. Elsedimy, A. Ismail, A. A. Salama, and M. Herajy, "DCNN: a novel binary and multi-class network intrusion detection model via deep convolutional neural network," EURASIP Journal on Information Security, vol. 2024, art. no. 36, Dec. 2024, doi: 10.1186/s13635-024-00184-1.

[28] P. K. Keserwani, M. C. Govil, E. S. Pilli, and P. Govil, "A smart anomaly-based intrusion detection system for the Internet of Things (IoT) network using GWO–PSO–RF model," Journal of Reliable Intelligent Environments, vol. 7, no. 1, pp. 3–21, Jan. 2021, doi: 10.1007/s40860-020-00126-x.

[29] F. S. Alsubaei, "Smart deep learning model for enhanced IoT intrusion detection," Scientific Reports, vol. 15, art. no. 6363, 2025, doi: 10.1038/s41598-025-06363-5.

[30] N. Dash, S. Chakravarty, A. K. Rath, N. C. Giri, K. M. AboRas, and N. Gowtham, "An optimized LSTM-based deep learning model for anomaly network intrusion detection," Scientific Reports, vol. 15, art. no. 1554, Jan. 2025, doi: 10.1038/s41598-025-85248-z.

[31] S. Kaushik, A. Bhardwaj, A. Almogren, S. Bharany, A. Altameem, A. U. Rehman, S. Hussen, and H. Hamam, "Efficient, lightweight cyber intrusion detection system for IoT ecosystems using mi2g algorithm," Scientific Reports, vol. 15, art. no. 88286, 2025, doi: 10.1038/s41598-025-88286-9.

[32] P. Waghmode, M. Kanumuri, H. El-Ocla, and T. Boyle, "Intrusion detection system based on machine learning using Least Square Support Vector Machine," Scientific Reports, vol. 15, art. no. 12066, Mar. 2025, doi: 10.1038/s41598-025-95621-7.

- [33] S. Almahaqeri, M. H. Almourish, A. A. Nasser, A. S. A. Alghawli, A. A. K. Elsayed, and A. N. Alhejoj, "A multi-class classification model on CICIOT2023 using Kolmogorov–Arnold Networks and tree ensembles," *Scientific Reports*, vol. 16, art. no. 47399, Apr. 2026, doi: 10.1038/s41598-026-47399-5.
- [34] A. Jyoti, M. Singh, V. K. Banga, J. Kaur, J. Gowrishankar, S. Hota, S. Murugan, A. Bhowmik, and N. Ashok, "Comparative performance evaluation of machine learning classifiers for multi-class intrusion detection on the NSL-KDD dataset," *Scientific Reports*, 2026, doi: 10.1038/s41598-026-59610-8.
- [35] T. B. Ogunseyi and G. Thiyagarajan, "An Explainable LSTM-Based Intrusion Detection System Optimized by Firefly Algorithm for IoT Networks," *Sensors*, vol. 25, no. 7, art. no. 2288, Apr. 2025, doi: 10.3390/s25072288.